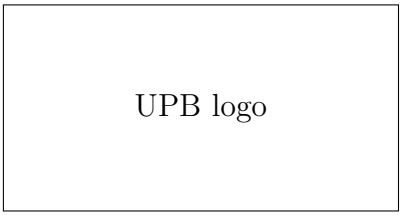
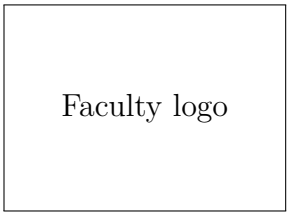
UNIVERSITY POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING
DEPARTMENT





DIPLOMA PROJECT

Scheduling de Task-uri În Sisteme Distribuite
Workflow-aware scheduling for heterogeneous Kubernetes clusters

Bogdan Dumitrescu

Thesis advisor:

Prof. dr. ing. Radu-Ioan Ciobanu

BUCHAREST

2026

Sinopsis

Această lucrare studiază planificarea task-urilor unui workflow (graf aciclic orientat, DAG) pe un cluster eterogen: ce task gata de execuție rulează următorul și pe ce nod. Am construit două suprafețe de execuție care folosesc aceleași descrieri de clustere și workflow-uri: un simulator cu evenimente discrete, care modelează CPU cu SMT, cache și memorie, stocare, rețea rutată cu partajare max-min și limite TCP, încălzire și throttling termic, energie, defecte, checkpoint-uri, gang-uri și preempțiune, respectiv un plan de control Kubernetes (CRD-uri Workflow și TaskTemplate, controller de DAG, scheduler propriu, date locale producătorului, telemetrie de nod). Un audit al versiunii anterioare a găsit 53 de probleme, dintre care unele schimbau rezultatele; toate au fost rezolvate, aproape toate prin corecturi de cod acoperite de teste. Pe baza unui eșec măsurat al lui HEFT (estimări care presupun transferuri fără contenție) propunem HEFT-RA, o variantă cu estimări conștiente de contenția rețelei, de imaginile partajate, de încălzire și de defecte, plus o anticipare pe un nivel. Pe 3.750 de cazuri holdout noi, HEFT-RA are regret mediu de 0,37% și P95 de 0,00%, față de 12,42% și 49,02% pentru HEFT; scorarea implicită Kubernetes are 160,69%. Pe clustere Kind care emulează aceleași familii, o variantă HEFT-RA a fost cea mai rapidă dintre nouă politici pentru DAG-uri concurente în 8 din 9 rulări, iar politicile care ignoră clasele de prioritate întârzie task-urile urgente. Validarea live a scos la iveală și corectat un defect al scheduler-ului, iar un “geamăn” calibrat al simulatorului reproduce timpii live cu o eroare medie de 3–12% după modelarea latenței de eliberare a sloturilor.

Abstract

This thesis studies how to schedule the tasks of a workflow, a directed acyclic graph (DAG) of dependent tasks, on a heterogeneous cluster: which ready task should run next, and on which node. Two execution surfaces share one set of cluster and workflow descriptions. The first is a discrete-event simulator that models CPUs with simultaneous multithreading, caches and memory bandwidth, storage, routed networks with max-min sharing and TCP limits, heating and thermal throttling, energy, failures, checkpoints, gangs, and preemption. The second is a Kubernetes control plane made of Workflow and TaskTemplate custom resources, a DAG controller, a custom scheduler, a producer-local data plane, and node telemetry.

An audit of the earlier version of the project found 53 problems, several of which changed results; all were resolved, almost all by code fixes covered by regression tests. A measured failure of HEFT, whose estimates assume that transfers never compete for a link, motivated HEFT-RA: HEFT's task order with finish-time estimates that account for network contention, shared image pulls, heating, and expected failure loss, plus a one-level lookahead. On 3,750 fresh holdout cases HEFT-RA has 0.37% mean and 0.00% P95 makespan regret, against 12.42% and 49.02% for HEFT and 160.69% mean regret for a model of the default Kubernetes scoring. With a capacity-aware lookahead it also leads the overlapping-workload experiments on median and tail response regret and deadline hit rate, while a preemptive priority policy remains the specialist for urgent bursts. On same-host Kind clusters that emulate the same cluster families, a HEFT-RA variant was the fastest of nine policies for concurrent DAGs in 8 of 9 runs (makespan 0.917 of HEFT's), and policies that ignore priority classes delay urgent work. The live validation also exposed and fixed a scheduler defect that froze nodes for up to 30 seconds, and a calibrated simulator twin reproduces live times within 3–12% mean absolute error (3–8% on the heterogeneous clusters) once the measured slot-release latency is modelled.

Contents

Sinopsis **1**

Abstract **2**

1 Introduction **6**

1.1 Context 6
1.2 Problem 6
1.3 Motivation: why Argo and kube-scheduler are not enough 7
1.4 Objectives 7
1.5 Research questions and hypotheses 8
1.6 Proposed solution 9
1.7 Contributions 9
1.8 A note on the history of this work 10
1.9 Structure of the thesis 10

2 State of the Art **11**

2.1 Workflow (DAG) scheduling on heterogeneous resources 11
2.2 The workflow-to-scheduler interface 12
2.3 Gang co-scheduling for fork-join stages 12
2.4 Admission control and queueing 12
2.5 Data locality versus image locality 13
2.6 Cold starts and image distribution 13
2.7 Modelling heterogeneous hardware for simulation 13
2.8 Learning and adaptive scheduling 14
2.9 Positioning of this work 14

3 Architecture and Implementation **16**

3.1 Overview 16
3.2 Model: workflow, task, and signals 16
3.3 The discrete-event simulator 18
3.3.1 Event loop 18
3.3.2 Processors, memory, and storage 18

3.3.3 Network . . . 18
3.3.4 Heat, power, energy . . . 19
3.3.5 Failures, checkpoints, gangs, priorities . . . 19
3.3.6 Uncertainty and seeds . . . 19
3.3.7 Lower bound . . . 20
3.4 The workflow-to-scheduler interface . . . 20
3.5 Workflow-aware policies . . . 20
3.5.1 Controls and the default-Kubernetes model . . . 20
3.5.2 EFT and HEFT . . . 21
3.5.3 HEFT-RA . . . 21
3.5.4 HEFT-RA with a capacity-aware lookahead . . . 22
3.5.5 The Legacy scorer . . . 22
3.5.6 Adaptive . . . 23
3.6 Cold-start-aware scheduling . . . 23
3.7 Stage-level gang co-scheduling . . . 23
3.8 Learning loop: profiles, coverage, exploration . . . 23
3.9 Admission, priorities, and preemption . . . 24
3.10 The live control plane . . . 24
3.11 Same-host emulation of heterogeneous clusters . . . 25
3.12 The calibrated simulator twin . . . 26

4 Experimental Evaluation 27
4.1 Experimental setup . . . 27
4.2 Scenarios . . . 27
4.3 Compared variants . . . 28
4.4 Metrics . . . 28
4.5 Protocol . . . 29
4.6 Hypotheses tested . . . 30
4.7 Why HEFT fails: a fan-out case . . . 31
4.8 Closed DAG results . . . 31
4.9 Overlapping workloads . . . 34
4.10 Mechanism studies: cold starts and priorities . . . 35
4.11 Learning, tuning, and portfolios . . . 37
4.12 Live validation on Kubernetes . . . 37
4.13 Answers to the research questions . . . 40
4.14 Recommendations per workflow and cluster type . . . 42
4.15 Threats to validity . . . 44

5 Limitations and Future Directions 46

6 Conclusions **48**

A Corrections relative to the earlier version **50**

B Reproducing the results **52**

Chapter 1

Introduction

1.1 Context

Scheduling in a distributed system is the repeated decision of *when* and *where* a unit of work runs, under resource limits and under objectives such as latency, throughput, utilisation, and fairness. Two properties of modern clusters make this decision harder than it looks.

First, clusters are heterogeneous. A cluster may combine nodes optimised for CPU throughput, nodes with wide memory buses, nodes with fast local storage, and small edge devices connected through Wi-Fi or a wide-area link. The same task can take a few seconds on one node and several times longer on another, and moving its input data can cost more than running it.

Second, many applications are not single jobs but *workflows*: several stages connected by data dependencies, naturally modelled as a directed acyclic graph (DAG). In Kubernetes [14], workflow engines such as Argo Workflows [10, 11] decide which task becomes eligible once its parents finish, but the decision of where each Pod runs is made by kube-scheduler [15, 16], one Pod at a time, without knowledge of the DAG or of the workflow's end-to-end goal.

1.2 Problem

We consider an online stream of workflow executions on a heterogeneous Kubernetes cluster. The number of workflow *types* is limited, but each type recurs many times, and several instances can run concurrently. The problem therefore has two levels:

- **per workflow**: minimise the end-to-end time (makespan) of each instance, using DAG context such as the critical path and avoiding unnecessary data movement and start-up delays;

- **per system**: when many instances run at the same time, improve mean and tail response time and throughput, under contention and under priorities between workflows.

The central question is: *how should the tasks of a workflow be ordered and placed on heterogeneous nodes so that end-to-end latency is minimised, respecting compatibility constraints and contention?*

Following the original project description we separate two notions that share the word "locality". *Data locality* is about where a task's inputs and outputs are, and what it costs to move them along the DAG's edges [31, 32]. *Image locality* is about whether a container image is already present on a node, which affects start-up latency [17]. The project models both, which turned out to be essential (Chapter 4).

1.3 Motivation: why Argo and kube-scheduler are not enough

Argo executes a DAG according to its dependencies but does not optimise placement. kube-scheduler places Pods using resource requests and declarative rules (affinity, taints, tolerations) and scores nodes with plugins such as `NodeResourcesFit`, `BalancedAllocation` and `ImageLocality` [17]. It has no access to:

- the role of a task in the DAG (its criticality, its position on the critical path);
- the end-to-end objective (the workflow's makespan rather than a good local placement);
- processor speed differences between nodes, the location and size of the task's inputs, and the current state of links and nodes.

Local decisions can therefore lengthen the critical path, move data that need not move, and introduce avoidable cold starts. Our experiments quantify this: a faithful model of the default scoring has a mean makespan regret of 162.56% on our holdout cases (Section 4.8).

1.4 Objectives

1. Define an executable model for the online execution of many DAG workflow instances on a heterogeneous cluster, detailed enough to reproduce the effects that decide placement quality (processor speed, memory and cache behaviour, storage, network contention, heating, failures, start-up cost).

2. Design a workflow-to-scheduler interface that carries DAG structure, priority, gang membership, and data provenance to the scheduler.

3. Implement and compare workflow-aware placement policies, from simple controls to HEFT-style list scheduling, against a model of the default Kubernetes behaviour.

4. Implement an online learning loop (profiles of runtime and start-up per task template and node, with exploration) for recurring workflows.

5. Study system-level behaviour under concurrency: priorities, preemption, and their cost to other workflows.

6. Validate the simulator's conclusions on a real Kubernetes control plane running emulated heterogeneous clusters, and recommend a policy per workflow type and per cluster type.

1.5 Research questions and hypotheses

RQ1 Can a workflow-aware scheduler that uses DAG signals reduce makespan compared with local, Pod-by-Pod placement?
H1: list scheduling with upward ranks (HEFT family) has lower makespan regret than the default Kubernetes scoring.

RQ2 With many concurrent workflow instances, does the approach also improve system-level performance (mean and tail response, deadlines)?
H2: the policy that is best for single workflows is also best, or close to best, for overlapping workloads.

RQ3 How much does cold-start-aware placement (image locality, shared pulls, pre-warming) contribute to time-to-running and makespan?
H3: image-aware estimates reduce time-to-running; the remaining cold-start cost is bounded by a pre-warmed cluster.

RQ4 Does online learning (explore/exploit) make recurring workflows cheaper to schedule well?
H4: warm profiles lead to better placements than cold profiles, and tuning the scorer helps most when profiles are cold.

RQ5 Do priorities and preemption reduce the degradation of critical workflows without severely harming the others?
H5: preemption shortens urgent response at a small cost to non-urgent work.

A sixth, methodological question runs through the whole thesis: **RQ6** — *do conclusions drawn in simulation survive on a real control plane?*

1.6 Proposed solution

The solution adds a workflow-aware layer to Kubernetes and pairs it with a simulator that can evaluate thousands of scheduling situations:

- **Workflow and TaskTemplate custom resources** carry the DAG, priorities, gang sizes, and data contracts; a controller creates Pods only for ready tasks and annotates them with DAG context.
- **A custom scheduler** orders ready Pods and scores nodes with DAG ranks, learned runtime profiles, measured bandwidths, data provenance, and image presence; it binds Pods through the API and can preempt lower-priority work.
- **A producer-local data plane**: outputs stay on the node that produced them and are fetched by consumers, so data locality has a real cost.
- **HEFT-RA**, a resource-aware variant of HEFT, designed after the simulator exposed why HEFT fails on contention-heavy cases.
- **A same-host emulation** of heterogeneous clusters on Kind, and a **calibrated simulator twin** of each emulated cluster, to test whether simulated conclusions hold on the live system.

1.7 Contributions

1. A validated, deterministic discrete-event simulator for DAG scheduling on heterogeneous clusters, with directional and half-duplex links, TCP loss and window limits, SMT-aware CPU capacity, cache-aware memory traffic, an RC thermal model, energy, independent failure streams, gang-atomic recovery, and a control-plane delay.
2. HEFT-RA, a weight-free extension of HEFT whose estimates account for network contention (in-flight and same-round flows), shared image pulls, thermal throttling, and expected failure loss, with a one-level lookahead that, in its final form, only counts on capacity the children will actually find.
3. A multi-phase evaluation (exploratory, confirmatory, holdout, and a fresh holdout for the final policy set) with per-seed perturbation of every hardware and network parameter, which fixes the non-independence of seeds in the earlier evaluation.
4. A corrected Kubernetes control plane (53 audited findings, resolved with code fixes and regression tests or, for packaging issues, with corrected documentation), live implementations of all compared policies, and an emulation of heterogeneous

clusters on one PC with calibrated speeds, CPU-time work budgets, and shaped links.

5. Recommendations per workflow family and per cluster family, backed by holdout evidence and by live validation.

1.8 A note on the history of this work

An earlier version of this project reached different conclusions (HEFT as the best closed-DAG policy, Adaptive as the response-time specialist, and a Kubernetes comparison that favoured Adaptive). A systematic audit found that some of those conclusions rested on defects: a lower bound that was not a bound, random failures that depended on the scheduling policy, seeds that repeated the same experiment, and a live emulation in which every node ran tasks at the same speed. This thesis reports the corrected system and results; Appendix A lists what changed and why, and the engineering journal in the repository tells the story in detail.

1.9 Structure of the thesis

Chapter 2 reviews DAG scheduling, cluster schedulers, gang scheduling, admission control, locality, cold starts, and the performance models the simulator builds on. Chapter 3 describes the architecture: the model, the simulator, the policies (including HEFT-RA), the Kubernetes control plane, the emulation, and the twin. Chapter 4 presents the experimental protocol and results, both simulated and live, and answers the research questions. Chapter 5 discusses limitations and future work, and Chapter 6 concludes.

Chapter 2

State of the Art

2.1 Workflow (DAG) scheduling on heterogeneous resources

Scheduling a DAG on a set of processors to minimise makespan is NP-hard even in simple settings [2], so practical schedulers are heuristics. The most studied family is *list scheduling*: tasks are given priorities, then repeatedly the highest-priority ready task is assigned to the processor that optimises a cost function. Kwok and Ahmad [3] survey the classic static algorithms.

HEFT (Heterogeneous Earliest Finish Time) [1] is the reference algorithm for heterogeneous processors. Each task's *upward rank* is its average execution time plus the maximum, over its children, of the average communication time to the child plus the child's rank; the rank therefore approximates the length of the remaining critical path. Tasks are processed in decreasing rank order and each is placed on the processor with the earliest estimated finish time (EFT), including the time to receive its inputs. HEFT is simple, fast, and hard to beat consistently, which is why it is the standard baseline.

HEFT's weaknesses are well documented. Its placement is greedy, so a task may be put where it finishes early but its children cannot. PEFT uses an optimistic cost table to look ahead [4], and Bittencourt et al. [5] evaluate each placement by the finish times of the task's children, a one-level lookahead that we adopt. HEFT also assumes a *contention-free* communication model: every transfer is estimated as if it had the link to itself. Sinnen and Sousa [6] showed that ignoring contention can make schedules far longer than predicted and proposed contention-aware list scheduling over an explicit network graph. This thesis meets the same problem in a measured form (Section 4.7).

For workflows in clouds and clusters, Rodriguez and Buyya [7] classify algorithms by objective (makespan, cost, deadline), by when decisions are taken (static, dynamic, hybrid), and by the resource model. Characterisation studies of real scientific

workflows [8, 9] identify recurring structures (pipelines, fan-out/data distribution, fan-in/aggregation, diamonds) that our workflow corpus reproduces.

2.2 The workflow-to-scheduler interface

A scheduler can only be workflow-aware if someone tells it about the workflow. Lehmann et al. [12, 13] propose the Common Workflow Scheduler Interface (CWSI), a contract through which workflow engines expose DAG structure and task metadata to resource managers, so that scheduling can be separated from orchestration. In Kubernetes, the natural vehicles for such metadata are custom resources (CRDs) [18] and annotations, consumed either by a plugin of the scheduling framework [16] or by a secondary scheduler selected through `schedulerName`. This thesis uses CRDs for workflows and templates, and Pod annotations for the information the scheduler needs (DAG, ranks, gang size, priority, data provenance).

2.3 Gang co-scheduling for fork-join stages

When the tasks of a stage synchronise, starting only some of them wastes resources and lets stragglers lengthen the critical path. Gang scheduling [23] admits a group of tasks together or not at all. Kubernetes has long relied on external projects for this (Volcano [24], the co-scheduling plugin); recent releases add a Workload API and gang scheduling support [21, 20, 22]. Our simulator and live scheduler treat a gang as a single scheduling unit: it is placed only if all members fit, and in the simulator a failure of any member restarts the whole gang.

2.4 Admission control and queueing

In a shared cluster, *which* workflow is admitted can matter as much as where its next Pod runs. Kueue [25, 26] separates admission (quotas, fair sharing, queueing) from Pod placement. Classic cluster managers combine admission and placement with fairness policies: Borg [27] uses priorities and quotas, Omega [28] lets several schedulers share state optimistically, Sparrow [29] schedules short tasks with sampling, and Dominant Resource Fairness [30] generalises max-min fairness to multiple resources. Our live engine admits workflows into a queue ordered by priority and arrival and uses Kubernetes-style priority classes; quota-based admission is left for future work.

2.5 Data locality versus image locality

Data-intensive schedulers try to run tasks close to their data. Delay scheduling [31] waits briefly for a slot on a node that holds the input; Quincy [32] formulates locality and fairness as a min-cost flow. For DAGs, the cost of moving data along an edge is part of the finish-time estimate itself, as in HEFT. Image locality is a different effect: whether a container image is cached on the node. kube-scheduler's `ImageLocality` plugin favours nodes that already hold the image, scaled by image size and by how widely the image is spread [17]. Packing schedulers such as Tetris [33] show that multi-resource packing, rather than per-task placement, can also reduce job completion times; this is relevant to the one case where our list schedulers lose to simple packing (Section 4.8).

2.6 Cold starts and image distribution

For short tasks and latency-sensitive pipelines, start-up overhead (image pull, container creation, cold caches) can dominate. Lazy pulling, for example the stargz snapshotter [34], starts a container before its image is fully downloaded, and peer-to-peer distribution such as Dragonfly [35] spreads the load of image downloads. Our model represents images explicitly (size, registry link, per-node cache) and lets several tasks on a node share one pull.

2.7 Modelling heterogeneous hardware for simulation

Running thousands of scheduling experiments requires a simulator, and a simulator is only useful if it captures the effects that decide placement quality. The models used here come from well-established work:

- **Processors.** Execution time follows calibrated per-core throughput, Amdahl's law for the parallel fraction [36], and a roofline-style bound when memory traffic dominates compute [37]. Simultaneous multithreading adds only a fraction of a core's throughput [38]; caches absorb part of the memory traffic depending on the working set and its reuse [39].
- **Networks.** Flow-level simulators such as SimGrid [40] share link capacity between active flows by max-min fairness [41]; Velho et al. [42] showed that such models are accurate for bulk transfers once contention and round-trip effects are modelled. A single TCP flow is further limited by its window divided by the round-trip time and, under random loss, by the bound of Mathis et al. [43]; modern congestion control such as CUBIC [44] does somewhat better on long, lossy paths.

- **Heat and power.** A lumped thermal resistance/capacitance model, as in HotSpot [45], gives an exponential approach to a steady-state temperature that depends on power; above a threshold the processor throttles. Server power is well approximated as idle power plus a term linear in utilisation [46]. Temperature-aware placement has been studied at data-centre scale [47].
- **Failures.** Field studies show that failure rates differ strongly between nodes and that failures are not always independent [48]; we use per-node Poisson failures plus scripted fault injection. Checkpointing bounds the work lost to a failure [49].

For the live emulation we rely on Linux control groups (CPU quotas and weights) [50] and on `tc netem` [51] for link delay and rate, inside Kind [52], which runs Kubernetes nodes as containers.

2.8 Learning and adaptive scheduling

When execution times are not known in advance, schedulers learn them. Bandit algorithms such as UCB [54] balance exploring uncertain options and exploiting good ones; the project's earlier Adaptive policy uses a UCB-style optimism term over learned profiles. Deep reinforcement learning has been used to learn DAG scheduling policies for data-processing clusters (Decima [55]). Such policies need large amounts of training data and are hard to inspect; for a thesis whose aim is to explain *why* a policy wins, we prefer analytical estimators with explicit models and use learning only for the quantities that must be measured (runtimes, start-up times, bandwidths).

2.9 Positioning of this work

Table 2.1 summarises how the elements above are combined. The distinctive points are (i) a weight-free, contention- and thermal-aware HEFT variant evaluated against the components it builds on, (ii) an evaluation protocol that separates development, confirmation, and holdout data and perturbs every seed, and (iii) live validation on a real control plane through emulated heterogeneity and a calibrated twin.

Aspect	Typical prior approach	This work
DAG awareness in Kubernetes	Orchestrator decides readiness; kube-scheduler places Pods locally	CRDs carry the DAG; scheduler uses upward ranks
Communication model	Contention-free (HEFT) or explicit network graph (Sinnen)	Routed flows with max-min sharing and TCP limits, in both simulator and estimator
Heterogeneity in tests	Real clusters, or VMs that share one host	Fluid simulator plus Kind emulation with speed factors and shaped links
Start-up cost	ImageLocality score	Image size, registry path, per-node cache, shared pulls
Evaluation	Single seed range, aggregate tables	Exploratory, confirmatory, untouched holdout; perturbed seeds; paired contrasts

Table 2.1: Positioning of this work.

Chapter 3

Architecture and Implementation

3.1 Overview

The system has two execution surfaces that read the same kind of description and implement the same policies (Figure 3.1). The *simulator* answers comparative questions cheaply: thousands of (cluster, workflow, policy, seed) runs per minute, deterministically. The *Kubernetes control plane* answers whether the ideas work on a real API server, kubelets, cgroups, and network stack. A *twin* connects them: the simulator is calibrated on a short warm-up of the emulated cluster and then predicts the live trials.

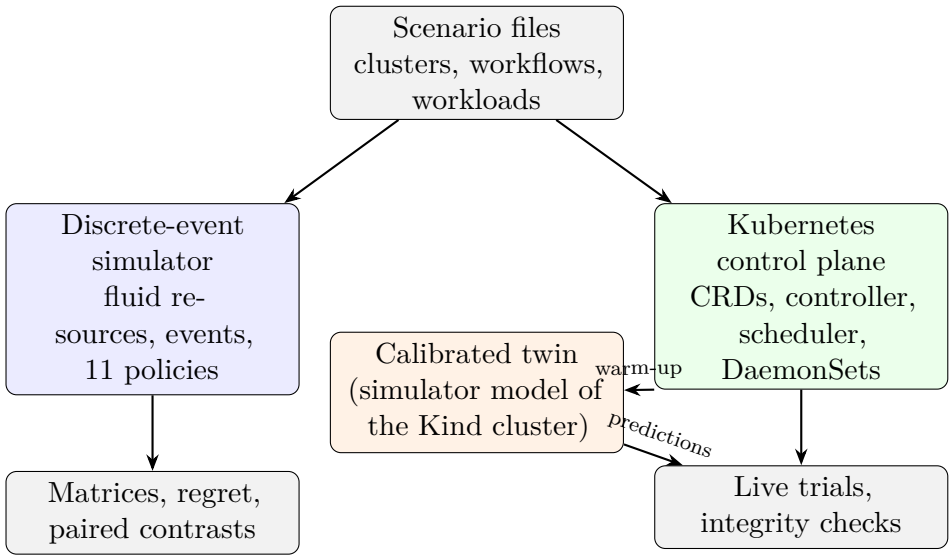


Figure 3.1: The two execution surfaces and the twin that connects them.

3.2 Model: workflow, task, and signals

All scenarios are YAML files validated by `simulation/schema.py`.

Nodes. A node has a CPU (architecture, physical cores, threads per core, base and maximum frequency, calibrated GFLOP/s per core, L3 size, idle and maximum power, SMT efficiency), memory (capacity, calibrated bandwidth, latency, channels, optional per-task bandwidth limit), storage (sequential and random read/write rates, optional per-task limits), a thermal description (ambient, initial, throttle and shutdown temperatures, thermal resistance and capacitance, minimum throttle factor), optional reliability parameters (Poisson failure rate, scripted failure times, mean repair time), and background load. Table 3.1 shows an example.

Links. A link connects two vertices (nodes, switches, or the image registry) and has bandwidth, latency, jitter, a TCP loss-event rate, a medium (`ethernet`, `wifi`, `wan`), an optional reverse bandwidth and latency for asymmetric links, and a duplex mode (Wi-Fi defaults to half duplex). Routes are shortest paths over this graph.

Tasks. A task declares its compute work (GFLOP) and parallel fraction, memory reads and writes with a working-set size and cache-reuse fraction, disk reads and writes with a random-I/O fraction and block size, CPU cores and memory requested, its container image and image size, start-up time, supported architectures, priority class, deadline, preemptability, checkpoint interval, release time, and an optional gang group. Dependencies carry the number of MiB that flow along the edge.

Workloads. A workload is a list of workflow submissions with arrival times, priorities, preemptability, and deadlines; it is compiled into a single DAG whose task identifiers are prefixed by the submission.

	`cpu-fast`	`memory-wide`	`io-fast`	`arm-edge`
Cores / threads	8 / 16	6 / 12	4 / 8	6 / 6
GFLOP/s per core	22	10	9	7
L3 (MiB)	32	48	12	8
DRAM (MiB/s)	28,000	72,000	26,000	15,000
Seq. read (MiB/s)	1,800	2,000	7,000	900
Power idle/max (W)	18 / 125	14 / 85	11 / 60	4 / 24
Throttle at	90 °C	88 °C	86 °C	78 °C
Link	238 MiB/s	238 MiB/s	238 MiB/s	60 MiB/s Wi-Fi, 3.5 ms

Table 3.1: The `balanced-heterogeneous-4` cluster.

3.3 The discrete-event simulator

3.3.1 Event loop

The engine (`simulation/engine.py`) keeps virtual time and jumps from event to event: task releases, the end of an image pull or input transfer, the end of an execution, a node reaching its throttle temperature, failures and repairs, and checkpoints. Between events, every active execution and transfer progresses at a constant rate; whenever the set of activities changes, all rates are recomputed. At each scheduling point the policy receives a read-only snapshot (ready tasks, node states, temperatures, warm images, running tasks, transfers in flight) and returns placements. A placed task first pulls its image if the node does not have it, then receives its inputs, then executes.

3.3.2 Processors, memory, and storage

A task's execution time on a node is the maximum of four independent ceilings, in the spirit of the roofline model [37]:

$$t_{\text{exec}} = \max\left(\frac{W}{r_{\text{cpu}}},\ \frac{M \cdot m(S,\rho,L3)}{B_{\text{dram}}},\ \frac{D_r}{B_{\text{read}}},\ \frac{D_w}{B_{\text{write}}}\right), \tag{3.1}$$

where W is compute work, r_{cpu} the task's CPU rate, M its memory traffic, m the fraction of that traffic that misses the L3 cache (a function of working set S, reuse ρ and cache size), and D_r, D_w its disk traffic. The CPU rate follows Amdahl's law for the requested cores and the task's parallel fraction, is scaled by the thermal throttle factor, and is limited by the node-wide capacity, which includes the SMT gain: a node with p physical cores of g GFLOP/s and SMT efficiency e offers at most $p\,g\,(1+e)$ when all hardware threads are busy. Concurrent tasks share CPU, DRAM bandwidth and storage by progressive filling, with optional per-task limits. Random I/O is limited by IOPS times block size.

3.3.3 Network

Each transfer is a fluid TCP flow along its route. A hop maps to a *channel*: one per direction on full-duplex media, one shared channel on half-duplex media. Rates are allocated by progressive max-min filling [41] over channels, and each flow is additionally limited by its receive window divided by the round-trip time and by the Mathis bound [43]

$$r \le \frac{MSS\,\sqrt{3/2}}{RTT\,\sqrt{p}}, \tag{3.2}$$

where p is the combined loss-event rate along the route. A protocol efficiency of 0.97 accounts for header overhead, and per-transfer jitter is drawn from the transfer's own random stream. Image pulls are flows from the registry vertex.

3.3.4 Heat, power, energy

Each node follows a first-order RC model [45]: with power P, ambient T_a, resistance R and capacitance C, the temperature approaches $T_a + PR$ exponentially with time constant RC. Power grows from idle to maximum with the share of busy hardware threads along a mildly convex curve (exponent 1.4), a refinement of the near-linear relation reported for servers [46]. Above the throttle temperature the CPU rate is reduced linearly towards the minimum throttle factor at the shutdown temperature. The engine computes the exact time at which a node will cross its throttle point and schedules an event there, so throttling starts at the right moment rather than at the next periodic step. Energy is the integral of power over time.

3.3.5 Failures, checkpoints, gangs, priorities

Nodes fail according to a Poisson process at their declared rate and/or at scripted times, and are repaired after a sampled delay. Each node draws its failures from its own random stream, derived from the run seed and the node identifier, so that two policies run on the same seed experience exactly the same failures. A failure interrupts the node's tasks; a checkpointable task resumes from its last checkpoint, others restart. A gang is scheduled only when all its members can start, and a failure of any member restarts the whole gang. Priority classes (from `BEST_EFFORT` through `BATCH` and `REAL_TIME_MEDIUM` to `CRITICAL`) order tasks for priority-aware policies, and preemptive policies may interrupt preemptable lower-priority tasks. A cluster-level control-plane delay models the time between a task becoming ready and becoming placeable.

3.3.6 Uncertainty and seeds

A seed controls (i) a perturbation of the cluster's hardware and network parameters, (ii) jitter, and (iii) random failures. Each cluster declares a calibration uncertainty u; the experiment runner can impose a floor on it. Every node's per-core throughput, DRAM bandwidth and latency, storage rates and IOPS, idle and dynamic power, thermal resistance and capacitance, failure rate and repair time, and every link's bandwidth, latency and jitter are then multiplied by independent seeded factors drawn uniformly from $[1-u, 1+u]$; structural parameters (core counts, cache sizes, temperature thresholds, loss rates) are unchanged. With a floor of 0.1, every seed is therefore a distinct instance of the declared cluster. Section 4.5 explains why this floor was necessary.

3.3.7 Lower bound

For sanity checks the runner computes a lower bound on any achievable makespan: the maximum of the critical path (each task on its fastest compatible node, unthrottled, with a private L3) and four aggregate terms (total compute over total SMT-aware CPU capacity, total DRAM traffic after cache hits over total DRAM bandwidth, total reads and writes over total storage rates). Every term is optimistic, so the bound is valid.

3.4 The workflow-to-scheduler interface

Two custom resources define the live API (`crds/`). A **TaskTemplate** describes a reusable task: image and pull policy, task class (CPU-, memory-, I/O-bound), logical CPU and memory request, optional physical resources and emulation scales, compatible node types, command, arguments, environment, expected output bytes, checkpointability, and gang group. A **Workflow** lists tasks (each referring to a template) with typed dependencies, plus a priority, a workflow class, and preemptability.

The controller creates a Pod only when all of a task's parents have succeeded, sets `schedulerName` to the custom scheduler, and passes the context the scheduler needs as annotations: the compact DAG of the whole workflow (tasks, templates, typed edges with expected bytes), the task's class, compatible node types, logical requests, priority, preemptability, gang group and gang size, and the producer node and size of every input. Tasks receive environment variables for their identity, their input and output directories, their node name (downward API), and the emulated speed table (an optional ConfigMap).

3.5 Workflow-aware policies

3.5.1 Controls and the default-Kubernetes model

FCFS places each ready task on the first node that fits; Random picks a seeded random feasible node. `k8s-default` models the default kube-scheduler profile: Pods are taken in PrioritySort order (priority, then readiness time) and placed on the feasible node with the highest sum of NodeResourcesFit (LeastAllocated), NodeResourcesBalancedAllocation, and ImageLocality scores, with the default thresholds of 23 MiB and 1000 MiB for image size and the spread scaling, and random tie-breaking. It sees requests and image presence only; it is blind to processor speed, data location, temperature and failures.

3.5.2 EFT and HEFT

EFT takes ready tasks in readiness order and places each on the node with the earliest estimated finish time. HEFT [1] orders ready tasks by upward rank

$$\mathrm{rank}_u(t) = \overline{w}(t) + \max_{c \in \mathrm{succ}(t)} \left(\overline{c}(t, c) + \mathrm{rank}_u(c) \right), \tag{3.3}$$

where $\overline{w}$ is the task's average execution time over nodes and $\overline{c}$ the average communication time of the edge. Both use the same estimator: start-up (including an image pull if needed), the slowest input transfer at the route's uncontended rate, and the execution time on that node next to the tasks already running there.

3.5.3 HEFT-RA

HEFT-RA keeps HEFT's order and replaces its estimate. It was designed after the simulator showed that HEFT's contention-free estimate fails badly on data-heavy fan-outs (Section 4.7). Its estimate of the finish time of task t on node n differs in four ways:

1. **Contended transfers.** Every input transfer receives a max-min share of each channel on its route, given the transfers already in flight *and* those planned earlier in the same scheduling round, and is capped by the TCP window and loss bounds.

2. **Shared image pulls.** If the image is already being pulled to n, or a pull was planned there in this round, the task does not pay for a second pull.

3. **Thermal prediction.** CPU work is integrated over the RC model for the predicted duration, so a node that will heat past its throttle point during a long task is charged for the slowdown.

4. **Expected failure loss.** On a node with failure rate λ, a task of duration d fails with probability $1 - e^{-\lambda d}$; the expected delay (half the exposed work, at most half a checkpoint interval for checkpointable tasks, plus the restart cost) is added.

Finally, a producer is scored with a one-level lookahead [5]: for every child, the best (contended transfer + execution) over all nodes is computed with the siblings pulling from the producer concurrently, and the producer's score is the latest child finish. Listing 3.1 summarises one scheduling round. HEFT-RA has no weights or tuned parameters; any difference from HEFT is due to the estimator.

```
flows   <- transfers in flight;  pulls <- image pulls in flight
for unit in scheduling_units(ready tasks sorted by upward rank):
    for each task t in unit, for each feasible node n:
        own  <- start-up(t, n, pulls)
```

```
            + contended_transfer(inputs(t) -> n, flows)
            + thermal_execution(t, n, co-runners)
            + expected_failure_delay(t, n)
      look <- max over children c of t:
                own + min over nodes m of
                    contended_transfer(n -> m, siblings) + exec(c, m
                       )
      score(t, n) <- max(own, look)
  place the unit on the best feasible assignment (all-or-nothing)
  flows <- flows + planned input flows;  pulls <- pulls + planned
     pulls
```

Listing 3.1: One HEFT-RA scheduling round.

3.5.4 HEFT-RA with a capacity-aware lookahead

The first live trials showed a weakness of the lookahead that the wide simulated nodes had hidden: it assumes that every child can run on its best node, even when that node offers only one or two task slots and other work is about to take them. `heft-ra-cap` keeps everything else in HEFT-RA and changes only the lookahead. The children of a producer are assigned the way the non-delay list scheduler will actually place them: each goes to the best node that still has a free slot in the capacity left after the current round's commitments; a child waits for an earlier sibling only when every node is full; and only when no node has any capacity does the estimate fall back to the optimistic bound. Because the lookahead now needs the children's resource requests, which the live scheduler sees only once their Pods exist, a child's template is looked up in any workflow that used it, and otherwise is assumed to be as large as its producer. On the simulated clusters, whose nodes run 8 to 16 tasks at a time, both variants make identical decisions for single workflows (Section 4.8).

3.5.5 The Legacy scorer

The project's original live scheduler scores each feasible node with a weighted sum of eight normalised factors: resource fit, node availability, load balance, data locality, type affinity, warm image, failure penalty, and memory pressure. `legacy-v1` uses the original weights; `legacy-tuned` uses weights found by a seeded random search (256 log-normal candidates at a fixed absolute scale of 125, with training, validation and holdout splits and guardrails on retries, deadlines and P95 makespan). Both enforce three observations per compatible node before exploiting, and explore at random 10% of the time afterwards. `legacy` is a historical simulator approximation kept for continuity.

3.5.6 Adaptive

`adaptive` orders tasks by priority class and then by upward rank, and scores nodes by the EFT estimate plus penalties for energy (0.002 s/J), transfer volume (0.001 s/MiB), thermal pressure (up to 10 s), and failure probability (100 s times the probability). It may preempt lower-priority preemptable work for urgent tasks. `adaptive-no-preempt` is identical with preemption disabled. Live, Adaptive also adds a UCB-style optimism term over learned runtimes [54].

3.6 Cold-start-aware scheduling

Images are first-class in both surfaces. In the simulator a cold start is a flow of the image's size from the registry over the routed network, shared fairly with data flows; a node caches every image it has pulled. EFT, HEFT and Adaptive include the pull in their estimate; HEFT-RA also avoids paying twice for a pull already in progress; `k8s-default` uses ImageLocality. The live scheduler learns start-up time per template and node in its profiles, which includes the pull when the image is absent. Section 4.10 measures how much of the makespan is due to cold starts and how much image awareness recovers.

3.7 Stage-level gang co-scheduling

Both surfaces group ready tasks of the same gang into one scheduling unit. A unit is placed only if every member fits (the simulator searches member assignments with branch and bound; scores are shifted to be non-negative so that pruning on partial sums is valid), otherwise all members wait. Placement reservations of a unit are rolled back if any member cannot be placed. In the simulator, a node failure that interrupts one member interrupts the whole gang.

3.8 Learning loop: profiles, coverage, exploration

The live scheduler keeps a profile per (task template, node): a rolling window of runtime and start-up observations whose median is used as the estimate, an exponential moving average ($\alpha = 0.3$) used to detect drift, a count, and failures. Profiles are persisted and restored without replaying observations over the stored average. *Coverage*: until a template has three observations on every compatible node, the least sampled node is chosen; afterwards policies exploit. Legacy keeps 10% random exploration; Adaptive adds UCB optimism ($\beta = 1$). The simulator gives the Legacy variants seeded profiles

(12 observations per task and node, with the cluster's uncertainty) and compares cold and warm profiles in the weight search.

3.9 Admission, priorities, and preemption

The live engine admits workflows into a queue ordered by priority class and arrival, and orders ready tasks by effective priority and enqueue time; a `REAL_TIME_MEDIUM` task that waits too long is promoted (aging). Each tick (1 s) the policy receives the ready Pods, the cluster state, and the profiles, and returns placements, holds, and preemptions. No policy overcommits a node; a CRITICAL task that does not fit waits until resources are released. Preemption is explicit kill-and-restart, as in Kubernetes' own Pod preemption [19]: a victim must be strictly lower priority and preemptable, a per-node budget limits how many victims (including ones already planned in the same tick) a node can lose in a window, and a victim is never chosen twice. A preempted task enters a dedicated `PREEMPTED` state and is re-queued only when the controller has created its replacement Pod, which prevents the double dispatch found in the earlier version.

3.10 The live control plane

Controller. Watches Workflows and Pods, creates Pods for ready tasks with collision-free names (a readable prefix plus a hash of workflow, task and UID), propagates failures to descendants, retries failed tasks, and records per-task timestamps (ready, Pod created, started, finished, attempts) and the admission and finish time of each workflow. Task output is parsed strictly; a malformed output marks the task failed instead of wedging it.

Scheduler. Watches pending Pods that name it, builds task entries from annotations (including upward ranks computed from the published DAG), runs the policy each tick, and binds through the API. A bind has three outcomes (success, retry, drop) and the reservation is rolled back on anything but success. Cluster state is refreshed from the API outside the scheduling lock and merged with the scheduler's own reservations, so a refresh cannot erase decisions that the API has not yet reflected. The merge also skips Pods whose completion or preemption the scheduler has already processed: the inventory is polled outside the lock, so a Pod that finished between the poll and the merge still looks running in it, and charging it again froze its node until the next refresh 30 seconds later (Section 4.12). One lock protects all shared state.

Data plane and telemetry. Outputs are written under the producing node's `/var/lib/ts-data` and served by a per-node fileserver; a task fetches its parents' outputs at start and reports how long that took. A bandwidth probe DaemonSet measures transfers from every peer in staggered time slots and annotates nodes with the measured rates; a thermal DaemonSet reports real sensor temperatures where they exist.

3.11 Same-host emulation of heterogeneous clusters

The validation clusters run on one laptop (Intel Core i5-8300H, 4 cores / 8 threads, 12 GB RAM [53]) under Docker Desktop. Virtual machines were rejected: they share the same four cores, so a "fast" VM cannot be faster than the host, and each would cost gigabytes of memory. Instead, `kind_lab.py` builds a Kind cluster from a profile in `clusters/kind_profiles.yaml` and emulates heterogeneity in four ways:

1. **Slots.** Every validation task requests and is limited to 500m CPU, so a worker with k slots runs k tasks at a time.

2. **Container caps.** Each worker container gets $0.5k + 0.35$ CPUs (the extra 0.35 for node agents), $224k + 576$ MiB of memory, and CPU shares proportional to k.

3. **Speeds.** A ConfigMap gives each node a CPU, memory and I/O speed. CPU and memory tasks receive a *CPU-time budget* and keep working until they have consumed budget/speed CPU-seconds; I/O tasks write real bytes at a paced rate scaled by the I/O speed.

4. **Links.** `tc netem` [51] on each worker's egress sets delay and rate; since outputs stay on their producer, the producer's egress shapes every transfer it serves.

CPU-time budgets replaced an earlier design that counted iterations. Calibration showed that the same number of iterations ran at 0.99–1.09 million hashes per second on single-slot nodes but only 0.63 million on a two-slot node, because concurrent tasks shared physical cores through SMT and containers shared the host by cgroup weight [50]: the "fast" node was slower per task than the "slow" one. Consumed CPU time is insensitive to this effect, so a task's duration depends only on its budget, its node's speed, and its quota. After the change, a CPU task on a node with CPU speed 0.45 reported 6.83 s for a 3.33 CPU-second budget, i.e. 3.07 s of speed-normalised work against 3.0 s expected.

3.12 The calibrated simulator twin

`simulation/kind_twin.py` turns an emulated cluster into a simulator cluster: each worker becomes a node whose cores equal its slots and whose per-class rates follow the speed table; each link becomes a directional link with the netem delay and the *measured* egress rate. The twin is calibrated only on the warm-up phase of each live run: the base work of each template is the median of (runtime − staging time) × speed, the start-up time is the median delay from bind to task start, and the control-plane delay is the mean delay from admission to Pod creation plus the median first-wave dispatch latency. A second variant also keeps a finished task's slot reserved for the *slot-release latency*: the median time, again measured on the warm-up only, from a task finishing on a node to the next bind on that node (2.25–2.75 s across the three profiles), which covers the kubelet's status update, the watch event, and the next scheduling tick. The twin then runs the same batches under the same policies and predicts each trial. Agreement between the twin and the live system is the evidence for RQ6.

Chapter 4

Experimental Evaluation

4.1 Experimental setup

All experiments ran on one laptop: Intel Core i5-8300H (Coffee Lake, 4 cores / 8 threads, 2.3–4.0 GHz, 1 MiB L2, 8 MiB L3, 45 W) [53], 12 GiB of DDR4 (4 GiB + 8 GiB modules at 2133 MT/s), a Samsung 970 EVO Plus NVMe SSD, Windows 11 Pro, Docker Desktop 29.6 with a WSL2 virtual machine of 8 virtual CPUs and about 5.7 GiB of memory, Kind 0.32 with Kubernetes 1.36.1, and Python 3.12 for the simulator and tooling (task images use Python 3.11). The laptop is also described as a simulated cluster (`local-i5-8300h`) derived from this inventory. Simulation and live timing never overlapped: CPU-heavy work was scheduled before or after live trials, because both share the same four cores.

4.2 Scenarios

Clusters. Table 4.1 lists the five simulated cluster families. Four are synthetic but physically plausible; the fifth describes the development laptop itself.

Workflows. Fifteen closed workflows cover the structures found in scientific and data-processing workflows [8, 9] and the resource classes of the problem statement (Table 4.2).

Workloads. Four overlapping workloads submit several workflows over time: `mixed-arrivals` (repeated mixed and topology workflows in overlapping bursts, including a critical one), `priority-burst` (CPU-saturating batch work interrupted after 2 s by a critical three-stage workflow with a 12 s deadline), `data-locality-wave` (overlapping pipelines that stress producer locality, cache warmth, storage and network), and `recovery-gang-mix` (checkpointable batch work and a gang, followed by a non-preemptable critical arrival with a deadline).

Cluster	Description
`homogeneous-4`	Four equal 8-thread nodes behind a non-blocking 1 Gb/s switch (119 MiB/s links). Control case.
`balanced-heterogeneous-4`	CPU-, memory-, and I/O-optimised nodes and a 6-core ARM edge node on 60 MiB/s Wi-Fi with 3.5 ms latency (Table 3.1).
`asymmetric-edge-5`	Two core nodes (476 MiB/s) and three edge nodes on Wi-Fi (60, 40, 25 MiB/s; 2.5–8 ms; loss 0.05–0.2%) sharing a 50 MiB/s, 12 ms WAN uplink to the core and the registry.
`unreliable-thermal-3`	A fast but poorly cooled node with scripted failures at 20 s and 75 s, a reliable well-cooled node, and an I/O node that fails about 8 times per hour on a lossy link.
`local-i5-8300h`	The development laptop as a single node (25% declared uncertainty).

Table 4.1: Simulated cluster families.

4.3 Compared variants

The variants map onto the comparison planned in the original project description:

- **Baseline**: `k8s-default` (the default kube-scheduler scoring), plus the controls `fcfs` and `random`.
- **Workflow-aware**: `eft`, `heft`, `heft-ra`, and `heft-ra-cap` (DAG ranks, finish-time placement, data and image awareness).
- **Project history**: `legacy` (historical simulator heuristic), `legacy-v1` (the original live scorer), and `legacy-tuned` (the same scorer with searched weights).
- **Priorities and preemption**: `adaptive` and `adaptive-no-preempt`.
- **Reference**: `oracle-static`, an exhaustive search over static assignments for workflows with few tasks; it is never counted as a competitor.

Cold-start awareness and gang scheduling are mechanisms present in several policies rather than separate policies; Section 4.10 isolates them.

4.4 Metrics

- **Makespan** of a closed workflow, and **response time** (finish minus arrival) of each workflow in a workload.
- **Regret** of a policy on a case: its makespan (or mean response) divided by the best deployable policy's on the same case, minus one. Reported as mean (with

Workflow	Family	Description
`linear-5`	linear	Five equal CPU stages in a chain.
`deep-10`	deep	Ten light stages exposing critical-path placement costs.
`diamond`	diamond	One producer, three parallel specialists, one reducer.
`wide-12`	wide	Twelve independent equal tasks (packing, contention).
`fanout-1to8`	fan-out	One I/O source sends 1 GiB to each of eight CPU workers.
`fanin-8to1`	fan-in	Eight producers feed one memory-heavy reducer.
`cpu-heavy-8`	cpu-heavy	Eight parallel compute tasks (packing and heat).
`memory-heavy-6`	memory-heavy	Six streaming tasks contending for DRAM.
`io-heavy-6`	io-heavy	Sequential and random storage tasks (throughput, IOPS).
`cache-sensitive`	cache-sensitive	Equal memory traffic, small vs large working sets.
`data-heavy-pipeline`	data-heavy	Modest compute, multi-gigabyte intermediates.
`mixed-etl`	mixed	Storage ingest, CPU and memory transforms, export.
`gang-4`	gang	Four workers that must start together.
`critical-deadline`	priority	Short critical chain with a deadline.
`checkpoint-long`	failure-recovery	Long checkpointable compute under failures.

Table 4.2: Closed workflow corpus.

a 95% bootstrap confidence interval [57]), median, and 95th percentile (P95), because tail behaviour is what users notice [56].

- **Win/tie rate**: share of cases with zero regret.
- **Paired ratio**: for two policies on identical cases, the mean of the per-case ratio, with a bootstrap interval and the counts of wins, ties, and losses.
- **Deadline hit rate**, **energy**, **data moved**, **preemptions**, **time-to-running**, and **cold starts** where relevant.

4.5 Protocol

Four phases. Seeds are split into an *exploratory* phase (0–19), used for all development including the design of HEFT-RA; a *confirmatory* phase (200–249), run once the code was frozen; and a *holdout* phase (400–449), which was not looked at before every choice had been made. The capacity-aware lookahead was added afterwards, in response to

the live trials, so the holdout had already been seen when it was designed; a second, fresh holdout (seeds 600–649) was therefore run once, for the final policy set, and is the primary evidence below. The weight search and the contextual search use their own disjoint seeds and their own train/validation/holdout splits.

Why every seed is perturbed. The earlier evaluation reported "20 seeds per case", but a seed only changes a run through sampled uncertainty, jitter, and random failures, and four of the five clusters declare no uncertainty. Counting distinct outcomes over the 20 exploratory seeds of each (cluster, workflow, policy) showed how few independent experiments there really were: 1.4 on average for the homogeneous cluster, 3.0 for the balanced one, 4.0 for the unreliable one and 4.3 for the edge cluster (only the laptop profile, with 25% declared uncertainty, had 20). Intervals computed as if those were 20 independent samples would have been far too narrow. The confirmatory and both holdout phases therefore impose an uncertainty floor of 0.1, which turns every seed into a distinct cluster instance within ±10% of the declared one; on the holdout every one of the 50 seeds gives a distinct outcome for every cluster, case, and policy. The exploratory phase keeps the declared values, so comparing phases also shows whether the conclusions survive the perturbation.

Scale. Table 4.3 gives the size of each phase. FCFS and Random were dropped after the exploratory phase because they were never competitive.

Phase	Seeds	Uncertainty	Policies	Closed records	Workload records
Exploratory	0–19	declared	12 + oracle	18,300	4,800
Confirmatory	200–249	floor 0.1	10	37,500	10,000
Holdout	400–449	floor 0.1	10	37,500	10,000
Fresh holdout	600–649	floor 0.1	10	37,500	10,000

Table 4.3: Simulation phases (5 clusters × 15 workflows or 4 workloads).

4.6 Hypotheses tested

H1 (DAG awareness helps) is tested by comparing the HEFT family with `k8s-default` and the controls on closed DAGs; H2 (the single-workflow winner also wins under overlap) by the workload matrices; H3 (cold-start awareness) and H5 (preemption) by the targeted studies of Section 4.10; H4 (learning) by the cold- and warm-profile evaluation of the Legacy scorer and by the live warm-up; RQ6 by the live matrix and the twin.

4.7 Why HEFT fails: a fan-out case

Before the aggregate results, one case explains most of what follows. On `balanced-heterogeneous-4`, `fanout-1to8` has a source that reads 4 GiB from disk and eight CPU workers that each need a 1 GiB copy of the source's output (Table 4.4).

Policy	Makespan	Data moved	Placement
FCFS	6.63 s	168 MiB	all nine tasks on `cpu-fast`
EFT, HEFT	40.42 s	8,360 MiB	source on `io-fast`, workers on `cpu-fast`
HEFT-RA without lookahead	12.94 s	1,288 MiB	source and seven workers on `io-fast`
HEFT-RA	6.63 s	168 MiB	all nine tasks on `cpu-fast`

Table 4.4: The fan-out case (seed 0). Data moved includes image pulls.

HEFT puts the source on `io-fast`, whose 7 GB/s drive finishes the read at 1.10 s instead of 2.79 s on `cpu-fast`, a locally correct choice. It then estimates each worker's finish time on `cpu-fast` as if that worker's 1 GiB transfer had the 238 MiB/s link to itself (about 8.7 s), so all eight workers choose `cpu-fast`. Their eight transfers then share the link: 8 GiB at 238 MiB/s takes 34.4 s, the workers start at 37.15 s, and the workflow ends at 40.42 s, six times later than FCFS. This is the contention-free assumption that Sinnen and Sousa [6] warned about, made concrete. HEFT-RA's contended estimates alone keep seven workers next to the data (12.94 s); its lookahead then moves the source itself to `cpu-fast`, where every transfer disappears (6.63 s).

4.8 Closed DAG results

Table 4.5 gives the ranking on the fresh holdout (seeds 600–649), with the first holdout (400–449) alongside. The ordering is by median regret, then P95, then mean; since most medians are zero, the P95 decides.

	Fresh holdout (600–649)				Holdout (400–449)	
Policy	Mean regret [95% CI]	Median	P95	Win/tie	Mean	P95
`heft-ra`	**0.37%** [0.28, 0.47]	0.00%	**0.00%**	**95.39%**	0.39%	0.12%
`heft-ra-cap`	**0.37%** [0.28, 0.47]	0.00%	**0.00%**	**95.39%**	0.39%	0.12%
`heft`	12.42% [10.57, 14.45]	0.00%	49.02%	80.05%	12.06%	50.59%
`eft`	12.54% [10.68, 14.57]	0.00%	49.02%	79.65%	12.23%	50.59%
`legacy`	13.61% [11.62, 15.77]	0.00%	54.23%	78.83%	13.39%	52.05%
`adaptive`	13.21% [11.83, 14.69]	0.00%	57.83%	71.95%	13.42%	64.20%
`adaptive-no-preempt`	13.21% [11.83, 14.69]	0.00%	57.83%	71.95%	13.42%	64.20%
`legacy-tuned`	18.34% [16.57, 20.15]	0.00%	73.00%	56.64%	18.42%	68.09%
`legacy-v1`	27.80% [25.35, 30.21]	0.00%	173.88%	52.91%	27.64%	175.47%
`k8s-default`	160.69% [144.58, 176.40]	5.20%	1296.03%	42.32%	162.56%	1257.52%

Table 4.5: Closed DAGs, holdout phases (3,750 cases each).

Several observations follow.

- **HEFT-RA is close to the best policy almost everywhere.** Its mean regret is 0.37% on the fresh holdout, and in 95% of cases it is the best policy or tied with it. The paired ratio against HEFT is 0.9602 [0.9561, 0.9642] with 712 wins, 3,015 ties and 23 losses (first holdout: 0.9604, 701 / 3,023 / 26).
- **The capacity-aware lookahead changes nothing for single workflows.** `heft-ra-cap` and `heft-ra` make identical decisions on all 3,750 cases of each holdout: simulated nodes run 8 to 16 tasks at once, so a producer's children always find capacity.
- **The order is stable across phases.** The confirmatory phase gives HEFT-RA 0.38% mean and 0.00% P95 regret and HEFT 12.31% and 49.91%; the exploratory phase (declared uncertainty, twelve policies) gives HEFT-RA 1.76% and 5.75% and HEFT 11.34% and 57.20%.
- **HEFT's large regret is mostly a change of reference.** Regret is measured against the best policy, which is now HEFT-RA. Without HEFT-RA in the set, HEFT's first-holdout regret is 2.44% mean and 9.44% P95, the best of the remaining policies; HEFT remains the best of the classic heuristics.
- **DAG awareness matters a great deal.** `k8s-default` has 160.69% mean regret; the paired ratio HEFT/`k8s-default` is 0.8133 [0.8039, 0.8230]. FCFS (502%) and Random (2,811%) in the exploratory phase show how large placement effects are.
- **Preemption is irrelevant for a single workflow**: `adaptive` and `adaptive-no-preempt` are identical; Adaptive's penalties make it worse than plain HEFT (paired ratio 1.0335).
- **Tuning the Legacy scorer helps** (`legacy-tuned` 18.34% vs `legacy-v1` 27.80%; paired ratio 0.9710) but cannot close the gap: a weighted sum of normalised factors does not estimate finish time.
- **Against the static oracle** (300 small exploratory cases), HEFT-RA has the smallest mean gap (2.53%, median 0%), followed by HEFT (3.06%) and EFT (3.85%).

Where HEFT-RA loses. On 17 of the 3,750 first-holdout cases HEFT-RA is more than 1% slower than HEFT, and all 17 are `unreliable-thermal-3`/`mixed-etl`. There, the fast node fails at a scripted time (20 s) that no policy can know. HEFT-RA correctly predicts that the ETL chain finishes earlier on that node (its aggregate step ends at 17.5 s instead of 24.5 s), but in some seeds the failure hits the final export, which must wait

for the repair. Over the 50 seeds of this pair HEFT-RA still wins 39 and loses 11 (mean ratio 0.878; median 33.9 s against 46.1 s). Measured against the best policy of each case, however, its mean regret in that cell is 18.18% on the fresh holdout, where Adaptive and tuned Legacy, which avoid the risky node, tie for the best. The losses are the price of a risk-neutral estimator. A second, structural limit is `homogeneous-4/fanin-8to1`, where packing beats the list schedulers because consolidating the eight producers avoids gathering their outputs; fixing it needs group-level packing [33] rather than a better per-task estimate, and we did not tune for it.

By workflow and cluster family. Table 4.6 lists the fresh-holdout leaders. HEFT-RA (with either lookahead) has the lowest mean regret in every workflow family, tied with other list schedulers in `failure-recovery`, `gang` and `wide`, and in every cluster family except the single-node laptop, where all policies are identical because there is only one place to run.

Workflow family	Lowest mean regret	HEFT-RA mean regret
cache-sensitive	`heft-ra`, `heft-ra-cap`	0.35%
cpu-heavy	`heft-ra`, `heft-ra-cap`	1.12%
data-heavy	`heft-ra`, `heft-ra-cap`	0.00%
deep	`heft-ra`, `heft-ra-cap`	0.02%
diamond	`heft-ra`, `heft-ra-cap`	0.00%
failure-recovery	tie: EFT, HEFT, both HEFT-RA, Legacy	0.00%
fan-in	`heft-ra`, `heft-ra-cap`	0.02%
fan-out	`heft-ra`, `heft-ra-cap`	0.00%
gang	tie: EFT, HEFT, both HEFT-RA	0.00%
io-heavy	`heft-ra`, `heft-ra-cap`	0.00%
linear	`heft-ra`, `heft-ra-cap`	0.00%
memory-heavy	`heft-ra`, `heft-ra-cap`	0.00%
mixed	`heft-ra`, `heft-ra-cap`	3.64%
priority	`heft-ra`, `heft-ra-cap`	0.43%
wide	tie: EFT, HEFT, both HEFT-RA, Legacy	0.00%
Cluster family		
asymmetric-network	`heft-ra`, `heft-ra-cap`	0.00%
balanced-heterogeneous	`heft-ra`, `heft-ra-cap`	0.03%
homogeneous	`heft-ra`, `heft-ra-cap`	0.14%
local-calibration	all ten policies tie	0.00%
unreliable-thermal	`heft-ra`, `heft-ra-cap`	1.69%

Table 4.6: Closed DAGs: fresh-holdout leaders by family (regret computed within each family).

Where another policy is better. Crossing the 15 workflow families with the 5 cluster families gives 75 cells. In 69 of them HEFT-RA is the best policy or tied with it on the fresh holdout. The six exceptions are listed in Table 4.7; only one of them is large.

Cluster family	Workflow family	Best policy (mean regret)	HEFT-RA
unreliable-thermal	mixed	Adaptive (both), tuned Legacy (0.00%)	18.18%
homogeneous	cache-sensitive	Legacy (0.43%)	1.74%
unreliable-thermal	priority	`k8s-default` (0.00%)	1.58%
balanced-heterogeneous	priority	`k8s-default` (0.18%)	0.38%
homogeneous	fan-in	Adaptive (both) (0.00%)	0.08%
homogeneous	io-heavy	EFT, HEFT, Legacy (0.00%)	<0.005%

Table 4.7: Closed DAGs: fresh-holdout cells in which HEFT-RA is not the best policy.

4.9 Overlapping workloads

Table 4.8 gives the fresh-holdout ranking by median response regret.

Policy	Median resp. regret	P95 resp. regret	Deadline hit	Median makespan
`heft-ra-cap`	**0.00%**	**63.76%**	**71.47%**	**47.21 s**
`heft-ra`	0.51%	**63.76%**	70.53%	48.20 s
`adaptive`	0.65%	83.76%	69.20%	53.28 s
`eft`	0.97%	82.56%	68.13%	53.10 s
`adaptive-no-preempt`	1.00%	83.76%	69.20%	50.91 s
`legacy`	1.05%	82.87%	67.73%	56.06 s
`heft`	1.29%	82.56%	68.13%	52.23 s
`legacy-tuned`	8.11%	98.70%	69.73%	57.56 s
`legacy-v1`	32.64%	214.11%	66.53%	69.15 s
`k8s-default`	86.62%	1420.09%	36.27%	115.43 s

Table 4.8: Overlapping workloads, fresh holdout (1,000 cases).

When several workflows compete for nodes, the two lookahead variants do differ: `heft-ra-cap` leads or ties on every column. Its paired mean-response ratio against `heft-ra` is 0.9973 [0.9951, 0.9995] (98 wins, 873 ties, 29 losses), a small but consistent gain; on the first holdout the same contrast is 0.9998 [0.9983, 1.0014], i.e. no measurable difference, and the ranking there is otherwise the same (`heft-ra-cap` 0.04% and `heft-ra` 0.42% median response regret). Against HEFT, `heft-ra-cap` has a ratio of 0.9617 [0.9494, 0.9735] (504 / 281 / 215) and `heft-ra` 0.9643 [0.9520, 0.9761]; HEFT against `k8s-default` is 0.6689 [0.6446, 0.6937]. Adaptive is third: priority ordering plus preemption helps response a little (0.9935 [0.9889, 0.9978] against Adaptive without preemption, 0.9671 [0.9555, 0.9789] against HEFT). By workload, `heft-ra-cap` leads `data-locality-wave`, `mixed-arrivals` and `priority-burst` on mean response regret and EFT and HEFT tie on `recovery-gang-mix`; on the first holdout Adaptive led `priority-burst`. Response regret averages over all workflows of a workload, so it can hide what happens to the one urgent workflow; Section 4.10 looks at that directly. The overlapping results are noisier than the closed ones (P95 response regrets above 60% for every policy) because a single early decision can delay a whole later submission.

Where HEFT-RA should not be used for overlapping work. The workload-by-cluster cross table shows that on two cluster families HEFT-RA is far from the best policy, and `heft-ra-cap` is not among the leaders there either. On the single-node laptop profile, where placement is trivial and only ordering matters, Adaptive leads `mixed-arrivals` (2.26% mean response regret against 29.55% for HEFT-RA) and `priority-burst` (0.00% against 64.12%), and tuned or original Legacy leads `data-locality-wave` (1.21% against 24.83%). On the unreliable cluster, Adaptive leads `mixed-arrivals` (0.66% against 38.77%) and EFT and Legacy lead `data-locality-wave` (15.29% against 42.32%). In both cases the winning policies order urgent work first or avoid the failure-prone node, which HEFT-RA's risk-neutral, priority-blind design does not.

4.10 Mechanism studies: cold starts and priorities

Two research questions concern mechanisms rather than whole policies. Both studies use the holdout protocol (seeds from 400, ±10% perturbation) and compare variants paired on identical cluster instances (`run_mechanism_studies.py`, `results/mechanism-studies.md`).

Cold starts (RQ3). All 15 closed workflows on all 5 clusters (seeds 400–409, 750 runs per variant) were run under HEFT-RA as is, under HEFT-RA whose estimates believe every image is already present (pulls still happen), under HEFT-RA on a cluster where every image is pre-pulled everywhere, and under `k8s-default`, whose only start-up signal is ImageLocality (Table 4.9).

Variant	Mean makespan	Mean time-to-running	P95 time-to-running	Cold starts
HEFT-RA	20.19 s	1.13 s	11.76 s	3.79
HEFT-RA, image-blind estimates	20.42 s	1.17 s	13.82 s	3.79
HEFT-RA, all images pre-pulled	19.54 s	0.82 s	11.76 s	0.00
`k8s-default`	31.87 s	2.96 s	66.10 s	3.89

Table 4.9: Cold-start study (750 runs per variant).

Image-aware estimates help, but only a little: against image-blind estimates, HEFT-RA's makespan ratio is 0.9879 [0.9821, 0.9932] and its time-to-running ratio 0.9846 [0.9753, 0.9927], with most of the gain on the unreliable cluster (0.9409) and almost none where links are fast. The remaining cold-start cost is larger than what placement can recover: pre-pulling every image would cut makespan by 7.8% (ratio 0.9223 [0.9156, 0.9290], never worse) and time-to-running by about half (0.5103). Against the default scoring, HEFT-RA lowers makespan to 0.7798 and time-to-running to 0.7351 of `k8s-default`'s. H3 therefore holds in its weak form: image awareness reduces time-to-running, but in these scenarios prefetching or lazy pulling [34, 35] would be the bigger lever.

Priorities and preemption (RQ5). The three workloads that contain an urgent `CRITICAL` workflow were run on all clusters with 50 seeds (750 runs per policy), and the response of the urgent workflow was separated from everyone else's (Table 4.10).

Policy	Urgent response	Other response	Urgent deadline hit	Preemptions per run
`adaptive`	**10.93 s**	56.93 s	67.60%	0.34
`adaptive-no-preempt`	15.51 s	56.86 s	67.60%	0
`heft-ra`	20.94 s	**55.20 s**	**69.47%**	0
`heft`	21.33 s	57.36 s	66.13%	0
`eft`	21.76 s	56.87 s	66.13%	0
`k8s-default`	42.25 s	76.97 s	35.20%	0

Table 4.10: Priority study: mean response of the urgent and of the other workflows.

Preemption does what it promises at almost no cost to the others. It is triggered in 100 of the 750 runs; in those, the urgent workflow's response is roughly halved (mean ratio 0.534), and over all runs Adaptive against Adaptive without preemption has an urgent-response ratio of 0.9379 [0.9236, 0.9512] (99 wins, 1 loss) and an other-response ratio of 1.0006 [1.0000, 1.0012]. The effect is concentrated where long batch work occupies the cluster when the urgent workflow arrives: on `priority-burst` the urgent workflow finishes in 12.5 s under Adaptive, 25.3 s without preemption, and 36.5 s under HEFT-RA (means).

The means hide that the two families win in different places (Table 4.11). In the *typical* case HEFT-RA serves the urgent workflow faster: its median urgent response is 7.6 s against 9.7 s for Adaptive, and per run it is faster more often in every workload (139 against 83 runs on `mixed-arrivals`, 99 against 73 on `priority-burst`, 138 against 6 on `recovery-gang-mix`). Its urgent deadline hit rate is the same or slightly higher (80% against 80%, 54% against 52%, and 75% against 71%); the runs in which only one of the two meets the 12 s deadline are knife-edge cases, decided by about a tenth of a second, and 22 of the 24 that favour HEFT-RA are on the homogeneous cluster. Adaptive's value is the *tail*: when the urgent workflow arrives behind long batch work, HEFT-RA's rank order lets it wait, and its P95 urgent response is 126.6 s; Adaptive's priority ordering alone cuts that to 75.4 s, and preemption to 21.0 s. Priority ordering also has a price: against HEFT-RA, Adaptive makes the other workflows 7.8% slower per run (ratio 1.0781 [1.0519, 1.1097]; 56.9 s against 55.2 s in mean). H5 holds: preemption bounds the urgent workflow's worst case with a negligible cost relative to the same policy without it, but a priority-first policy costs typical urgent latency and non-urgent work relative to a makespan-first one.

Policy	Median urgent resp.	P95 urgent resp.	Urgent deadline hit
`heft-ra`	**7.6 s**	126.6 s	**69.47%**
`adaptive-no-preempt`	9.7 s	75.4 s	67.60%
`adaptive`	9.7 s	**21.0 s**	67.60%

Table 4.11: Urgent workflow: typical case versus tail (750 runs per policy).

4.11 Learning, tuning, and portfolios

Weight search. The Legacy scorer's weights were searched with 256 seeded log-normal candidates at a fixed absolute scale (scaling all weights by a constant does not change any decision), selected on 90 training and 45 validation cases, and evaluated once on 75 holdout cases. After the simulator corrections the search selected the same vector as before. With warm profiles the tuned/original objective is 0.9759 (mean makespan 42.18 s to 41.80 s, data moved 434.7 to 272.5 MiB); with cold profiles it is 0.8601 (mean makespan 99.98 s to 54.89 s). Learning matters: the original scorer with cold profiles is more than twice as slow as with warm ones, and tuning helps most exactly when profiles are poor (H4).

Factor ablation. Removing one factor at a time on 750 cases (seeds 300–309) shows that type affinity dominates (without it the objective is 3.4 times worse), that data locality (+0.088) and load balance (+0.072) matter, and that resource fit and node availability have no practical effect. Warm image, failure penalty and memory pressure never change a decision in the broad corpus, but each one flips the decision in a controlled stress case, so they are kept.

Contextual portfolio. A separate search over 160 generated workflows in 20 (resource mix × topology) contexts tried to assign a specialist policy to each context, replacing the default only for a validation gain of at least 0.5% with no mean or P95 regression. It kept HEFT in 19 contexts and chose HEFT-RA for CPU-bound fan-outs. On 600 holdout cases, relative to HEFT (lower is better), HEFT-RA alone scores 0.9952, the portfolio 0.9977, Adaptive 0.9989, and EFT 1.0254: using HEFT-RA everywhere beats the portfolio, so no portfolio is recommended.

4.12 Live validation on Kubernetes

Protocol. Each of the three emulated profiles (Section 3.11) was created from scratch, its applied `tc` qdiscs, container caps and speed table were recorded, and the control plane was installed. After a bandwidth-probe round and the warm-up, 54 trials ran in a seeded random order: two batches, nine live policies, three repetitions, each trial

starting from the same warm profiles. The `priority` batch fills all six slots with six long best-effort tasks and, 7 s later, submits a two-task `CRITICAL` workflow and three batch workflows (20 tasks); the `dag` batch submits four DAGs at once: a pipeline with a 96 MiB producer, a fan-out whose producer sends 96 MiB to each of four consumers, a fork-join, and independent mixed tasks (17 tasks). All 162 trials passed the integrity gate: every task succeeded on the node its own metrics report, at that node's emulated speed; every consumer read exactly the bytes its producer wrote, locally when co-located and remotely otherwise; the observed qdiscs matched the profile; every trial ran the intended policy with no scheduler errors; and only Adaptive preempted.

Emulation fidelity. Calibrated on the warm-up, the speed-normalised base runtimes were 3.03–3.07 s for the CPU template (target 3.0 s), 10.02–10.06 s for the long CPU template (10.0 s), 3.01 s for I/O (3.0 s), 3.05–3.25 s for large I/O (3.0 s) and 4.16–4.43 s for memory (4.0 s; allocating the buffer adds a little). Measured egress was 6.8 MiB/s for the balanced edge worker shaped at 60 Mbit/s (7.15 MiB/s) and 4.5 MiB/s for the edge workers shaped at 40 Mbit/s (4.77 MiB/s).

A defect found by the live runs. The first complete live run showed Pods pending for more than 30 s while nodes sat idle, most often under HEFT-RA. The cause was a race in the scheduler's periodic refresh: the inventory is polled outside the scheduling lock, so a Pod that finished between the poll and the merge still looked running, was charged as "untracked", and made its node look full until the next refresh 30 seconds later. Locality-seeking policies suffered most, because the frozen node is exactly the one where a producer's children were meant to run. The merge now ignores Pods whose completion or preemption the scheduler has processed (with a regression test that reproduces the race), the harness now records scheduler-wide errors that carry no Pod name, and all live results below come from a complete rerun with the fix; the earlier data is kept only as evidence of the defect. The fix is visible in the raw timelines: measuring, in every trial without preemption, the longest period during which a Pod was pending while some slot was free, the longest such stall was 33–38 s before the fix (13 stalls longer than 10 s in 69 trials) and 10.3 s after it (2 such stalls in 144 trials). The typical longest stall of 4–6 s per trial is the normal hand-over of a slot (Section 3.12) and is the same in both runs.

Concurrent DAGs. Table 4.12 pools the nine trials of each policy over the three profiles. The robust live result is that HEFT-RA was the fastest policy: a HEFT-RA variant had the lowest `dag` makespan in 8 of the 9 (profile, repetition) runs, which would happen with probability of about 4×10^{-5} if the nine policies were interchangeable; the exception was one edge-profile repetition won by HEFT. The mechanism is the one seen

in simulation. On the balanced profile HEFT-RA placed the 96 MiB fan-out producer on the CPU-optimised node, which has both the fastest egress and the fastest CPU for its four CPU consumers, in all six of its trials, while HEFT left it on the memory-optimised node in all three; HEFT then spent up to 2.8 times as long staging inputs (12.2 s against 4.4 s summed over the batch). Paired over matched trials, HEFT-RA's makespan is 0.917 [0.837, 0.998] of HEFT's (8 wins, 1 loss). That interval barely excludes one, and HEFT's own live result was not stable: on the balanced profile its median was 39.4 s in the earlier run (which still had the refresh defect) and 46.0 s in the rerun, while HEFT-RA's was about 40 s in both, so the 8.3% figure should be read as "HEFT-RA was consistently among the fastest" rather than as a precise effect size. The other contrasts are not conclusive with nine pairs: HEFT against `kube-default` 0.930 [0.857, 1.006] (7 wins, 1 loss), HEFT against EFT 0.917 [0.824, 1.031]. Unlike in simulation, where EFT and HEFT are almost equivalent, EFT was the slowest live policy on the `dag` batch. The two lookahead variants are indistinguishable (`heft-ra-cap` / `heft-ra`: 1.002 [0.930, 1.074]). Per profile, `heft-ra-cap` is fastest on the balanced cluster (35.9 s, against 39.2 s for Adaptive without preemption, 40.0 s for `heft-ra` and 46.0 s for HEFT), `heft-ra` is fastest on the edge cluster (34.5 s against 36.7 s for HEFT), and on the homogeneous cluster HEFT-RA (28.3 s) and HEFT (28.9 s) are equivalent, as they should be when nodes are identical. Tuning the Legacy weights did not transfer: `legacy-tuned` is slower than `legacy-v1` on mean response (1.053 [1.021, 1.086]; on makespan 1.043 [0.997, 1.092]). Note that `kube-default` is the default-scoring model running inside the custom scheduler, so that every policy shares the same control path; the stock kube-scheduler binary was not part of the comparison.

	`dag` batch		`priority` batch			
Policy	Makespan	Mean resp.	Makespan	Mean resp.	Critical	Preemptions
`heft-ra`	**34.04**	28.36	47.52	34.09	23.60	0
`heft-ra-cap`	34.74	**27.47**	47.81	35.04	23.89	0
`heft`	36.67	28.06	47.61	34.93	23.73	0
`adaptive`	36.94	27.87	51.81	33.45	**15.31**	31
`legacy-tuned`	38.11	28.66	48.05	31.46	16.64	0
`adaptive-no-preempt`	38.93	28.46	49.02	32.12	16.65	0
`kube-default`	38.99	28.58	48.37	31.83	17.33	0
`legacy-v1`	39.30	28.45	48.05	31.52	16.59	0
`eft`	41.79	29.23	**47.27**	**31.08**	16.56	0

Table 4.12: Live results pooled over the three profiles: medians of nine trials per policy, in seconds; preemptions are totals over the nine trials.

Priorities under contention. On the `priority` batch makespans are within 4% of each other except for Adaptive, whose kill-and-restart preemptions (31 over nine trials) raised makespan by about 14% (1.139 [0.997, 1.285] against Adaptive without

preemption, not conclusive) and P95 response by 26% (1.258 [1.054, 1.458]). In return the critical workflow finishes 12% sooner (0.878 [0.754, 0.969], 8 wins and no losses). This batch is the tail case of the simulated priority study: the critical workflow arrives while long best-effort tasks fill every slot. The HEFT family orders ready tasks by upward rank alone, so the two short urgent tasks wait behind higher-ranked batch work: HEFT's critical response is 1.476 [1.245, 1.746] times `kube-default`'s (0 wins in 9), whose PrioritySort queue puts them first. In this live batch even first-come ordering (EFT, 16.56 s) serves the critical workflow almost as well as priority ordering (Adaptive without preemption, 16.65 s), so most of the live gap comes from ranking urgent work last, and preemption adds a further 12%. The live system therefore confirms the second simulated conclusion as well: makespan-optimal placement and priority-aware ordering are different goals, and urgent work arriving into a busy cluster needs the latter. The capacity-aware lookahead was marginally slower than the original on this batch (mean response 1.024 [1.001, 1.046], 2 wins and 7 losses), one of 45 contrast-metric pairs computed, so it is reported but not interpreted.

The twin. Table 4.13 compares the median live result of each policy with the twin's prediction. The original twin, which frees a slot the instant a task ends, was optimistic everywhere (mean bias -5% to -23%). Adding the slot-release latency measured on the warm-up removes most of that bias and brings the mean absolute error to 3–8% on the heterogeneous profiles and 6–12% on the homogeneous one. Rank agreement between policies is moderate on the heterogeneous profiles (Spearman 0.30–0.80) and absent on the homogeneous profile, where all policies are within run-to-run noise of each other. The twin predicts the best policy of the balanced `dag` batch (the HEFT-RA variants) but not of the edge `dag` batch, where both twins favour `kube-default` (35.6 s predicted with the release latency, 39.0 s measured), and on the `priority` batch it does not reproduce the HEFT family's penalty on urgent work. The remaining biases have known sources that the warm-up cannot calibrate: a DAG edge costs 0.4–0.9 s more control-plane latency than a root task (the 10th percentile from a parent's finish to its child's bind is 2.9–3.2 s, against a calibrated 2.1–2.7 s), and the warm-up has no edges.

4.13 Answers to the research questions

RQ1 Yes. In simulation, HEFT's makespan is 0.8133 of the default Kubernetes scoring's on the fresh holdout, and HEFT-RA's mean regret is 0.37% against 160.69%. Live, on four concurrent DAGs, a HEFT-RA variant was the fastest of nine policies in 8 of 9 runs and its pooled median makespan is 34.0 s against 39.0 s for `kube-default`; HEFT-RA is 0.917 [0.837, 0.998] of HEFT's makespan, while plain HEFT's advantage over `kube-default` (0.930 [0.857, 1.006]) is not

		Original twin		With slot-release latency		
Profile	Batch	MAPE	Bias	MAPE	Bias	Spearman
balanced	dag	6.8%	−6.0%	6.0%	−0.8%	0.62
balanced	priority	12.6%	−12.6%	3.1%	+0.0%	0.30
edge	dag	5.8%	−5.4%	4.4%	+0.0%	0.50
edge	priority	19.6%	−15.5%	6.4%	−0.3%	0.47
homogeneous	dag	17.8%	−17.8%	12.4%	−12.4%	0.15
homogeneous	priority	11.1%	−9.6%	6.2%	−3.2%	−0.19

Table 4.13: Twin agreement on makespan (medians of three trials per policy; MAPE and bias over the nine policies).

conclusive with nine pairs. H1 holds: strongly in simulation, and live for the contention-aware variant.

RQ2 Yes for aggregate response; urgent work needs a bounded tail. With overlapping workloads the capacity-aware HEFT-RA has the lowest median (0.00%) and tail (63.76%) response regret and the highest deadline hit rate (71.47% against 36.27% for `k8s-default`), so the single-workflow winner also wins on aggregate (H2). For the urgent workflow alone, HEFT-RA is faster in the typical case (median 7.6 s against 9.7 s for Adaptive) and meets the urgent deadline as often, but its rank order can leave urgent work behind long batch tasks: its P95 urgent response is 126.6 s against 21.0 s for Adaptive, and live, where the critical workflow always arrives into a full cluster, the HEFT family's critical response is 1.48 times `kube-default`'s.

RQ3 A little, and less than prefetching would. Image-aware estimates lower makespan by 1.2% and time-to-running by 1.5%; pre-pulling every image would lower them by 7.8% and about 49%. H3 holds in its weak form.

RQ4 Yes. With cold profiles the original Legacy scorer's mean makespan is 99.98 s against 42.18 s with warm profiles, and tuning its weights helps most when profiles are cold (objective 0.8601 against 0.9759 with warm profiles). The live warm-up gives every learning policy its profiles before the trials. The tuned weights, however, did not transfer to the live cluster (1.043 of `legacy-v1`'s makespan on the DAG batch), so weights tuned in simulation should be re-validated live. H4 holds for learning, not for transferring tuned weights.

RQ5 Yes, with a cost that depends on restart price. In simulation preemption is triggered in 100 of 750 runs, roughly halves the urgent response when it is, cuts the P95 urgent response from 75.4 s to 21.0 s, and costs the other workflows 0.06%. Live it shortens the critical response by 12% (8 of 9 paired trials), but

killed tasks restart from scratch, so P95 response grows by 26% and makespan by about 14% (interval includes no change). H5 holds.

RQ6 **Partly.** The main simulated finding survives on the live control plane: HEFT-RA was the fastest policy for concurrent DAGs, for the same reason (it keeps data-heavy producers where their consumers can run and pays less for staging), and the live system confirms the priority trade-off. The finer ordering of the other policies does not transfer reliably: EFT, almost equal to HEFT in simulation, was the slowest live policy, and HEFT's own live result varied between runs (39.4 s and 46.0 s median on the balanced profile). A calibrated twin reproduces live times within 3–8% on the heterogeneous profiles, but only after the Kubernetes slot-release latency is modelled, and its ranking of policies agrees only moderately with the live one (Spearman 0.30–0.80). Absolute simulated gains are larger than live gains, because the live tasks are short compared with control-plane latencies and every node has one or two slots.

4.14 Recommendations per workflow and cluster type

Table 4.14 condenses the evidence into a decision table. "HEFT-RA" means either lookahead variant. They are identical on single workflows; with overlapping workloads the capacity-aware variant (`heft-ra-cap`) is better on average (0.9973 [0.9951, 0.9995] of `heft-ra`'s mean response, though it loses in 29 of 1,000 cases), and in the live trials the two are indistinguishable. `heft-ra-cap` is the default because it removes a known estimation error.

Situation	Recommended policy	Evidence
Known DAGs on a heterogeneous cluster, no urgent work	HEFT-RA	Lowest mean regret in every workflow and cluster family (fresh holdout 0.37%); live 0.917 of HEFT on concurrent DAGs
Data-heavy, fan-out, and pipeline workflows; edge or asymmetric networks	HEFT-RA	Contention-aware transfers and lookahead: fan-out case 40.4 s → 6.6 s; 0.00% regret on the asymmetric family
Homogeneous cluster	HEFT or HEFT-RA (equivalent)	0.14% simulated regret; live 28.3 s and 28.9 s; for fan-ins, packing can win

Situation	Recommended policy	Evidence
Overlapping workloads without urgent work	HEFT-RA	Lowest median and P95 response regret, highest deadline hit rate
Urgent workflows that may arrive while long batch work fills the cluster, where the urgent worst case matters	Adaptive; Adaptive without preemption if restarts are expensive	P95 urgent response 21.0 s (75.4 s without preemption) vs 126.6 s for HEFT-RA; critical 15.3 s vs 23.6 s live; preemption raised live P95 response by 26%
Urgent workflows with spare capacity, where typical latency or deadline hits matter	HEFT-RA	Median urgent response 7.6 s vs 9.7 s for Adaptive; urgent deadline hit 69.47% vs 67.60%
Failure-prone or thermally constrained fast nodes, mixed pipelines	Adaptive or tuned Legacy (risk-averse)	HEFT-RA's risk-neutral choice: 18.18% regret on unreliable/mixed; 38–42% on unreliable workloads
Single node	Any placement; priority-aware ordering for overlapping work	All policies tie on closed DAGs; Adaptive leads overlapping work
Original Legacy scorer must be kept	Tuned weights, re-validated live	0.971 of `legacy-v1` in simulation; 1.043 live
Start-up latency matters	Pre-pull or lazy-pull images in addition	Placement recovers 1.2%; pre-pulling 7.8% makespan, 49% time-to-running
Default Kubernetes scoring	Avoid for DAG workflows, but keep its priority-first queue for urgent work	160.69% mean regret in simulation; live makespan 39.0 s vs 34.0 s for HEFT-RA; its PrioritySort order serves critical work 1.48× faster than rank order

Table 4.14: Recommended policy per workflow and cluster type.

4.15 Threats to validity

Construct validity. Makespan and response regret measure distance from the best *deployable* policy in our portfolio, not from the optimum; the static oracle shows that the optimum can be up to 30% better on some small cases. Energy and deadlines are reported but were not the selection criteria.

Internal validity. The simulator is deterministic for a seed, policies see identical clusters and identical failures on a seed, and the holdout data was not consulted during development. Two properties of the simulated comparison must be kept in mind. First, every policy plans with the exact parameters of the cluster instance it runs on; only transfer jitter and the timing of random failures are unknown to it. The perturbed confirmatory and holdout phases therefore test robustness across cluster instances, not robustness to wrong estimates. Second, HEFT-RA's estimator calls the simulator's own resource models (max-min channel sharing, TCP ceilings, the RC thermal model), and it was designed on exploratory seeds by the same person who wrote the simulator. Its near-zero simulated regret is therefore partly by construction and is best read as a best case for a scheduler whose estimates match reality; HEFT and EFT also plan with exact parameters, so the simulated gap to them measures what modelling contention is worth, not what it is worth with noisy estimates. The live trials, in which every policy works from learned runtimes and measured bandwidths, are the test of the latter: HEFT-RA remained the fastest policy there, but its advantage is known far less precisely (Section 4.12). The earlier evaluation's defects (Appendix A) are themselves a reminder that subtle implementation errors can change conclusions; every fix has a regression test.

External validity. The clusters are synthetic, the workflows are representative structures rather than traces, and the live clusters share one laptop: their "nodes" have no independent thermal or failure domains, their links are shaped rather than physical, and the host's CPU is shared through cgroups. The Mathis bound describes loss-based TCP; CUBIC or BBR would deliver more on long lossy paths. Failures in the simulator are Poisson or scripted, while real failures are bursty and correlated [48]. Conclusions should therefore be read as "under these modelled conditions, and confirmed on a real control plane with emulated heterogeneity", not as physical-cluster results.

Statistical validity. Bootstrap intervals assume cases are exchangeable; cases share clusters and workflows, so intervals are somewhat optimistic. Paired win/tie/loss counts, phase-to-phase stability, and the distinct-outcome check are reported alongside them for that reason. The live matrix has only three repetitions per profile, batch and policy (nine pooled trials per policy and batch), and the three repetitions of a profile share one

warm-up, so they are not independent with respect to calibration noise: HEFT's median on the balanced profile was 46.0 s in the final run but 39.4 s in the earlier run, even though that run still had the refresh defect, which could only slow it down. Differences of a few percent between live policies are therefore within run-to-run noise; only contrasts whose paired intervals exclude one are interpreted, and, of 45 contrast-metric pairs, a few would cross that line by chance. The capacity-aware lookahead was designed after the first holdout had been seen, which is why a second, fresh holdout was run for it.

Chapter 5

Limitations and Future Directions

Risk-neutral estimates. HEFT-RA minimises expected finish time. When a fast node fails at an unlucky moment, as in the `mixed-etl` case on the unreliable cluster, the expected-value choice loses. A risk-averse variant could minimise a high quantile of the finish time or add the variance of the failure delay; it should be evaluated on the same protocol before adoption.

Priority-aware HEFT-RA. The HEFT family orders ready tasks by upward rank only, which serves urgent workflows poorly (live critical response 1.48 times that of a priority-first queue). Ordering by priority class first and by rank within a class, while keeping HEFT-RA's placement, should combine Adaptive's urgent response with HEFT-RA's makespan; it is the most promising next policy to evaluate.

Group-level packing. No list scheduler in the portfolio consolidates a fan-in's producers to avoid gathering their outputs, which is why FCFS wins `homogeneous-4/fanin-8to1`. Packing-aware DAG scheduling in the spirit of Tetris or Graphene is a natural extension.

Admission control. The live engine orders admission by priority and arrival but has no quotas or fair sharing between tenants. Integrating with Kueue, or implementing DRF-style admission, would let the system protect throughput and fairness under sustained overload.

Scheduler integration. The scheduler is a separate process selected through `schedulerName`. A production implementation would be a plugin of the Kubernetes scheduling framework (Score, Permit for gangs, PostFilter for preemption) and could use the Workload API for gangs.

Live fidelity. Thermal behaviour and failures were not emulated live (all Kind nodes share one CPU package and one kernel), so HEFT-RA's thermal and failure terms were validated only in simulation. Links are shaped only on egress; congestion inside the shared Docker bridge is not modelled by the twin. A physical cluster running k3s [58] (scripts are provided) is the next validation step.

Learning. Profiles learn runtimes per template and node; they do not yet predict runtimes from input size or features, and drift detection only flags divergence. Learned runtime models and per-workflow adaptation remain future work, as does a learned scheduler evaluated with the same holdout discipline.

Multi-objective and multi-cluster. Energy, cost and fairness are measured but not optimised jointly, and scheduling across clusters or along an edge-cloud continuum is out of scope.

Chapter 6

Conclusions

This thesis set out to find out whether a workflow-aware scheduler can place the tasks of DAG workflows on heterogeneous clusters better than the default, Pod-by-Pod behaviour of Kubernetes, and which algorithm to use for which kind of workflow and cluster.

The first result is methodological. An audit of the earlier version of the project found that several of its conclusions rested on defects: a lower bound that was not a bound, random failures that depended on the policy, seeds that mostly repeated the same experiment, links that were half duplex for every medium, and a live emulation in which every node ran tasks at the same speed. A second, live-only defect, a refresh race that froze nodes for 30 seconds, was found only by running the full live matrix. None of these errors was visible in aggregate tables; each was found by following one concrete trace. The corrected simulator, three-phase protocol with a fresh holdout, emulated live clusters, and calibrated twin are the infrastructure on which the other results rest.

The second result is an algorithm. HEFT, the standard heterogeneous list scheduler, fails badly whenever many transfers share a link, because its estimates assume every transfer has the link to itself; in the clearest case it is six times slower than FCFS. HEFT-RA keeps HEFT's order and fixes the estimate (contended transfers, shared image pulls, thermal prediction, expected failure loss) and adds a capacity-aware one-level lookahead. It has no tuned weights. On 3,750 fresh holdout cases its mean makespan regret is 0.37% against 12.42% for HEFT and 160.69% for the default scoring, it leads every workflow and cluster family, and with overlapping workloads it leads median and tail response and deadline hits.

The third result is that the main simulated finding survives on a real control plane. On emulated heterogeneous Kind clusters, a HEFT-RA variant was the fastest of nine policies for four concurrent DAGs in 8 of 9 runs, for the same reason as in simulation: it kept the data-heavy producer where its consumers could run and spent less time staging inputs. Its makespan was 0.917 of HEFT's, although HEFT's own live result varied between runs and the finer ordering of the other policies did not transfer. The

live system also confirmed the main trade-off: urgent work arriving into a busy cluster needs priority-first ordering, and preemption buys about 12% faster critical response at the price of restarted work. The twin, once it modelled the measured slot-release latency of Kubernetes, reproduced live times within 3–8% on the heterogeneous profiles, which supports using the simulator for comparative questions while reminding that control-plane latencies must be part of the model.

The recommendation is therefore: use HEFT-RA (with the capacity-aware lookahead) for DAG workflows on heterogeneous clusters, especially when data moves between tasks or the network is uneven; it also serves urgent work well in the typical case. Where an urgent workflow may arrive while long batch work fills the cluster and its worst-case latency matters, use a priority-first policy such as Adaptive, with preemption only where restarts are cheap: it cuts the P95 urgent response from about two minutes to about twenty seconds in simulation. Prefer risk-averse placement on clusters whose fastest nodes are also the least reliable, and add image pre-pulling where start-up latency matters, because placement alone recovers little of it. The default Kubernetes scoring should not be used to place DAG workflows, although its priority-first queue is worth keeping. Physical multi-machine validation, a priority-aware and risk-averse HEFT-RA, and a scheduling-framework implementation are the natural next steps.

Appendix A

Corrections relative to the earlier version

Table A.1 lists the findings of the audit that changed results or conclusions. The engineering journal in the repository describes each one with a worked example.

Finding	Consequence	Correction
Lower bound charged all memory traffic to DRAM, ignored SMT, used possibly throttled temperatures	350 of 26,250 earlier records had makespan below the “bound”	Optimistic bound with cache-aware DRAM traffic, SMT-aware capacity, unthrottled CPUs
One random stream for failures, jitter and transfers	Policies were compared under different failures	Independent streams per node and per transfer
Most seeds identical (no declared uncertainty)	“20 seeds” were 1.5–4.7 experiments; intervals too narrow	Uncertainty floor 0.1 for confirmatory and holdout phases
Links half-duplex everywhere; loss scaled capacity by $(1 - p)$	Opposite flows competed; loss nearly harmless	Directional channels, half-duplex Wi-Fi, TCP window and Mathis bounds; loss rates recalibrated
SMT ignored at node saturation	SMT efficiency had no effect when busy	SMT-aware node capacity
Gang member restarted alone after a failure	Gang semantics violated	Gang-atomic restart

Finding	Consequence	Correction
Branch and bound pruned with negative scores	Optimal gang placements pruned	Non-negative score contract
`bool("false")` and falsy `BEST_EFFORT`	Flags and priorities silently flipped	Explicit parsing
HEFT contention-blind	40.4 s vs 6.6 s on the fan-out case	HEFT-RA (new policy), HEFT kept as baseline
Live: preempted task re-dispatched while its Pod terminated	Two Pods for one task	`PREEMPTED` state
Live: refresh restored full capacity	Overcommit after every refresh	Merge observed Pods with logical reservations
Live: failed bind treated as success	Workflows hung	Bind outcomes with rollback
Live: truncated Pod names	Different tasks aliased	Hashed DNS-1123 names
Live: every node ran tasks at the same speed	Live heterogeneity untestable	Speed factors with CPU-time budgets
Live: no DAG in the scheduler	HEFT impossible live	DAG annotation, live upward ranks
Live: logical vs physical bytes, 1 s clocks	Biased transfer costs and timings	Real-size data, task-reported timestamps
Live: concurrent bandwidth probes	Edge measured at 25–40% of its rate	Staggered probe slots
Live: refresh merge charged just-finished Pods as untracked	Nodes looked full for up to 30 s; locality-seeking policies hit most	Merge skips Pods the scheduler already released; full live rerun
Harness kept only error lines naming a Pod	Scheduler-wide errors were invisible to the integrity gate	Scheduler-wide errors recorded and gated
HEFT-RA lookahead ignored free capacity	Producer co-located with children that could not all run there	Capacity-aware lookahead (`heft-ra-cap`)

Table A.1: Corrections that changed results.

Appendix B

Reproducing the results

```
# tests
.\.venv\Scripts\python.exe -m pytest -q
# closed-DAG and workload matrices (one call per phase; see README)
python run_experiment_lab.py run --seeds 400:450 --sample-uncertainty
    `
     --uncertainty-floor 0.1 --policies <policy list> --output results/
        sim/...
python run_experiment_lab.py workload --seeds 400:450 --sample-
    uncertainty `
     --uncertainty-floor 0.1 --policies <policy list> --output results/
        sim/...
#   (the fresh holdout uses --seeds 600:650)
python analyze_final_evidence.py
python run_mechanism_studies.py
# live validation: creates each Kind profile, runs the warm-up and
    trials,
# predicts them with the twin, and writes the validation report
powershell -File scripts\run-kind-validation.ps1 -Profiles balanced,
    homogeneous,edge
# twin with the measured slot-release latency, then the report
python kind_twin_predict.py results/kind-validation/<profile>.json --
    slot-release measured
python analyze_kind_validation.py
```

Listing B.1: Main entry points (PowerShell, repository root).

The README lists the exact command line, seed range and policy set of every retained artifact.

Bibliography

[1] H. Topcuoglu, S. Hariri, and M.-Y. Wu, *Performance-effective and low-complexity task scheduling for heterogeneous computing*, IEEE Transactions on Parallel and Distributed Systems, 13(3):260–274, 2002.

[2] J. D. Ullman, *NP-complete scheduling problems*, Journal of Computer and System Sciences, 10(3):384–393, 1975.

[3] Y.-K. Kwok and I. Ahmad, *Static scheduling algorithms for allocating directed task graphs to multiprocessors*, ACM Computing Surveys, 31(4):406–471, 1999.

[4] H. Arabnejad and J. G. Barbosa, *List scheduling algorithm for heterogeneous systems by an optimistic cost table*, IEEE Transactions on Parallel and Distributed Systems, 25(3):682–694, 2014.

[5] L. F. Bittencourt, R. Sakellariou, and E. R. M. Madeira, *DAG scheduling using a lookahead variant of the Heterogeneous Earliest Finish Time algorithm*, in Proc. 18th Euromicro Conference on Parallel, Distributed and Network-based Processing (PDP), pp. 27–34, 2010.

[6] O. Sinnen and L. A. Sousa, *Communication contention in task scheduling*, IEEE Transactions on Parallel and Distributed Systems, 16(6):503–515, 2005.

[7] M. A. Rodriguez and R. Buyya, *A taxonomy and survey on scheduling algorithms for scientific workflows in IaaS cloud computing environments*, Concurrency and Computation: Practice and Experience, 29(8):e4041, 2017.

[8] S. Bharathi, A. Chervenak, E. Deelman, G. Mehta, M.-H. Su, and K. Vahi, *Characterization of scientific workflows*, in Proc. 3rd Workshop on Workflows in Support of Large-Scale Science (WORKS), 2008.

[9] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, *Characterizing and profiling scientific workflows*, Future Generation Computer Systems, 29(3):682–692, 2013.

[10] Argo Project, *Argo Workflows: the workflow engine for Kubernetes*, https://argoproj.github.io/workflows/, accessed February 2026.

[11] Argo Project, *Argo Workflows documentation: DAG*, https://argo-workflows.readthedocs.io/en/latest/walk-through/dag/, accessed February 2026.

[12] F. Lehmann, J. Bader, F. Tschirpke, L. Thamsen, and U. Leser, *How workflow engines should talk to resource managers: a proposal for a common workflow scheduling interface*, in Proc. IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid), 2023.

[13] F. Lehmann, J. Bader, L. Thamsen, and U. Leser, *The Common Workflow Scheduler Interface: status quo and future plans*, in Proc. SC'23 Workshops (SC-W), 2023.

[14] The Kubernetes Authors, *Kubernetes documentation*, https://kubernetes.io/docs/, accessed 2026.

[15] The Kubernetes Authors, *Kubernetes scheduler*, https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/, accessed 2026.

[16] The Kubernetes Authors, *Scheduling framework*, https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/, accessed 2026.

[17] The Kubernetes Authors, *Scheduler configuration (plugins, including ImageLocality)*, https://kubernetes.io/docs/reference/scheduling/config/, accessed 2026.

[18] The Kubernetes Authors, *Custom resources*, https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/, accessed 2026.

[19] The Kubernetes Authors, *Pod priority and preemption*, https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/, accessed 2026.

[20] The Kubernetes Authors, *Gang scheduling*, https://kubernetes.io/docs/concepts/scheduling-eviction/gang-scheduling/, accessed February 2026.

[21] The Kubernetes Authors, *Workload API*, https://kubernetes.io/docs/concepts/workloads/workload-api/, accessed February 2026.

[22] The Kubernetes Authors, *Kubernetes v1.35: introducing workload aware scheduling*, https://kubernetes.io/blog/2025/12/29/kubernetes-v1-35-introducing-workload-aware-scheduling/, 2025.

[23] D. G. Feitelson and L. Rudolph, *Gang scheduling performance benefits for fine-grain synchronization*, Journal of Parallel and Distributed Computing, 16(4):306–318, 1992.

[24] Volcano Project, *Volcano: a cloud native batch system*, https://volcano.sh/, accessed 2026.

[25] The Kubernetes Authors, *Introducing Kueue*, https://kubernetes.io/blog/2022/10/04/introducing-kueue/, 2022.

[26] Kueue Project, *Kueue concepts*, https://kueue.sigs.k8s.io/docs/concepts/, accessed February 2026.

[27] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, *Large-scale cluster management at Google with Borg*, in Proc. 10th European Conference on Computer Systems (EuroSys), 2015.

[28] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, *Omega: flexible, scalable schedulers for large compute clusters*, in Proc. 8th ACM European Conference on Computer Systems (EuroSys), pp. 351–364, 2013.

[29] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, *Sparrow: distributed, low latency scheduling*, in Proc. 24th ACM Symposium on Operating Systems Principles (SOSP), pp. 69–84, 2013.

[30] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, *Dominant resource fairness: fair allocation of multiple resource types*, in Proc. 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 323–336, 2011.

[31] M. Zaharia, D. Borthakur, J. Sen Sarma, K. Elmeleegy, S. Shenker, and I. Stoica, *Delay scheduling: a simple technique for achieving locality and fairness in cluster scheduling*, in Proc. 5th European Conference on Computer Systems (EuroSys), pp. 265–278, 2010.

[32] M. Isard, V. Prabhakaran, J. Currey, U. Wieder, K. Talwar, and A. Goldberg, *Quincy: fair scheduling for distributed computing clusters*, in Proc. 22nd ACM Symposium on Operating Systems Principles (SOSP), pp. 261–276, 2009.

[33] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, *Multi-resource packing for cluster schedulers*, in Proc. ACM SIGCOMM, pp. 455–466, 2014.

[34] containerd Project, *Stargz snapshotter: lazy pulling of container images*, https://github.com/containerd/stargz-snapshotter, accessed February 2026.

[35] Dragonfly Project (CNCF), *Dragonfly: P2P-based image and file distribution*, https://github.com/dragonflyoss/Dragonfly, accessed February 2026.

[36] G. M. Amdahl, *Validity of the single processor approach to achieving large scale computing capabilities*, in Proc. AFIPS Spring Joint Computer Conference, pp. 483–485, 1967.

[37] S. Williams, A. Waterman, and D. Patterson, *Roofline: an insightful visual performance model for multicore architectures*, Communications of the ACM, 52(4):65–76, 2009.

[38] D. M. Tullsen, S. J. Eggers, and H. M. Levy, *Simultaneous multithreading: maximizing on-chip parallelism*, in Proc. 22nd Annual International Symposium on Computer Architecture (ISCA), pp. 392–403, 1995.

[39] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2017.

[40] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, *Versatile, scalable, and accurate simulation of distributed applications and platforms*, Journal of Parallel and Distributed Computing, 74(10):2899–2917, 2014.

[41] D. Bertsekas and R. Gallager, *Data Networks*, 2nd ed., Prentice Hall, 1992.

[42] P. Velho, L. M. Schnorr, H. Casanova, and A. Legrand, *On the validity of flow-level TCP network models for grid and cloud simulations*, ACM Transactions on Modeling and Computer Simulation, 23(4), article 23, 2013.

[43] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, *The macroscopic behavior of the TCP congestion avoidance algorithm*, ACM SIGCOMM Computer Communication Review, 27(3):67–82, 1997.

[44] S. Ha, I. Rhee, and L. Xu, *CUBIC: a new TCP-friendly high-speed TCP variant*, ACM SIGOPS Operating Systems Review, 42(5):64–74, 2008.

[45] K. Skadron, M. R. Stan, W. Huang, S. Velusamy, K. Sankaranarayanan, and D. Tarjan, *Temperature-aware microarchitecture*, in Proc. 30th Annual International Symposium on Computer Architecture (ISCA), pp. 2–13, 2003.

[46] X. Fan, W.-D. Weber, and L. A. Barroso, *Power provisioning for a warehouse-sized computer*, in Proc. 34th Annual International Symposium on Computer Architecture (ISCA), pp. 13–23, 2007.

[47] J. Moore, J. Chase, P. Ranganathan, and R. Sharma, *Making scheduling "cool": temperature-aware workload placement in data centers*, in Proc. USENIX Annual Technical Conference, 2005.

[48] B. Schroeder and G. A. Gibson, *A large-scale study of failures in high-performance computing systems*, IEEE Transactions on Dependable and Secure Computing, 7(4):337–350, 2010.

[49] J. W. Young, *A first order approximation to the optimum checkpoint interval*, Communications of the ACM, 17(9):530–531, 1974.

[50] The Linux Kernel Documentation, *Control Group v2*, https://docs.kernel.org/admin-guide/cgroup-v2.html, accessed 2026.

[51] S. Hemminger, *Network emulation with NetEm*, in Proc. Linux Conference Australia (linux.conf.au), 2005.

[52] Kubernetes SIG Testing, *kind quick start*, https://kind.sigs.k8s.io/docs/user/quick-start/, accessed 2026.

[53] Intel Corporation, *Intel Core i5-8300H processor specifications*, https://www.intel.com/content/www/us/en/products/sku/134876/intel-core-i58300h-processor-8m-cache-up-to-4-00-ghz/specifications.html, accessed 2026.

[54] P. Auer, N. Cesa-Bianchi, and P. Fischer, *Finite-time analysis of the multiarmed bandit problem*, Machine Learning, 47(2–3):235–256, 2002.

[55] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, *Learning scheduling algorithms for data processing clusters*, in Proc. ACM SIGCOMM, pp. 270–288, 2019.

[56] J. Dean and L. A. Barroso, *The tail at scale*, Communications of the ACM, 56(2):74–80, 2013.

[57] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.

[58] K3s Project, *K3s: lightweight Kubernetes*, https://k3s.io/, accessed 2026.